JAVA OOP

my handwritten notebook · from class to enum

Name	_____
Subject	Object Oriented Programming in Java
Lectures	Kunal Kushwaha · Java OOP playlist · OOP 1 → OOP 7
Code	github.com/kunal-kushwaha/DSA-Bootcamp-Java > lectures/17-oop
Roll No.	_____ Section: ____

```
Class  ──→  Blueprint
  ↓ new
Object ──→  Real Thing
```

```
REMEMBER
class  = plan on paper
object = thing in heap
```

✍️ How to read these notes

red highlight → critical Java rule
yellow highlight → important concept
green highlight → syntax / code pattern
blue highlight → definition / term
★ must remember ✓ correct × wrong
≠ different ? common confusion
🧠 mental model 💡 intuition
⚠️ mistake 🎯 interview trap ☕ Java reality

🔴 MUST KNOW 🟠 HIGHLY IMPORTANT 🟡 GOOD TO KNOW

Every page follows the same loop → WHY? → real life → concept → Java → code → visualise → mistake → trap → practice → revise.
On screen, answers are folded under "☑ Answers" — try first, then open.

✦ INDEX ✦

S.No.	Name of the Topic	Page No.
1	THE STORY OF OOP	3
01	**INTRODUCTION TO OOP**	4
2	WHY OOP?	4
3	CLASS & OBJECT	5
4	CLASS vs OBJECT	6
5	REFERENCES, new & MEMORY	7
6	REFERENCE vs OBJECT	8
7	CONSTRUCTORS	9
8	THE this KEYWORD	10
9	WRAPPER CLASSES	11
10	THE final KEYWORD	12
11	GARBAGE COLLECTION	13
02	**PACKAGES · STATIC · INNER CLASSES · SINGLETON**	14
12	PACKAGES	14
13	STATIC	15
14	STATIC — WHO CAN TALK TO WHOM?	16
15	STATIC vs NON-STATIC	17
16	STATIC BLOCK & INITIALIZATION ORDER	18
17	WHY IS main() STATIC?	19
18	HOW System.out.println() WORKS	20
19	INNER CLASSES	21
20	SINGLETON CLASS	22
03	**THE FOUR PILLARS OF OOP**	23
21	THE 4 PILLARS	23
22	INHERITANCE	24
23	super & THE CONSTRUCTOR CHAIN	25
24	this vs super	26
25	TYPES OF INHERITANCE	27
26	POLYMORPHISM	28
27	METHOD OVERLOADING (compile-time)	28
28	METHOD OVERRIDING (runtime)	29
29	POLYMORPHISM: Parent obj = new Child()	30

S.No.	Name of the Topic	Page No.
30	OVERLOADING vs OVERRIDING	31
31	final & static IN INHERITANCE	32
32	ENCAPSULATION	33
33	ABSTRACTION	34
34	ENCAPSULATION vs ABSTRACTION	35
04	**ACCESS CONTROL · PACKAGES · OBJECT CLASS**	36
35	ACCESS MODIFIERS	36
36	THE protected NUANCE	37
37	BUILT-IN PACKAGES	38
38	THE Object CLASS	39
39	equals() + hashCode()	40
40	== vs .equals()	41
41	getClass() & instanceof	42
05	**ABSTRACT CLASS · INTERFACE · ANNOTATIONS**	43
42	THE MULTIPLE INHERITANCE PROBLEM	43
43	ABSTRACT CLASSES	44
44	ABSTRACT CLASS — THE RULES	45
45	INTERFACES	46
46	INTERFACE REFERENCES	47
47	EXTENDING INTERFACES	48
48	default & static METHODS (Java 8+)	48
49	NESTED INTERFACES	49
50	ABSTRACT CLASS vs INTERFACE	50
51	ANNOTATIONS	51
06	**GENERICS · LAMBDAS · EXCEPTIONS · CLONING**	52
52	BUILD YOUR OWN ArrayList	52
53	GENERICS	53
54	BOUNDS & WILDCARDS	54
55	COMPARING OBJECTS	55
56	COMPARABLE vs COMPARATOR	56
57	LAMBDA EXPRESSIONS	57
58	EXCEPTION HANDLING	58
59	throw vs throws	59
60	CHECKED vs UNCHECKED	60
61	OBJECT CLONING	61

S.No.	Name of the Topic	Page No.
62	SHALLOW COPY vs DEEP COPY	62
07	**COLLECTIONS · VECTOR · ENUMS**	63
63	WHY COLLECTIONS?	63
64	THE COLLECTIONS MAP	64
65	THE BIG FOUR — QUICK TOUR	65
66	COLLECTION vs MAP	66
67	VECTOR & SYNCHRONIZATION	67
68	ENUMS	68
69	ENUM CONSTRUCTORS, METHODS & INHERITANCE	69
★	**INTERVIEW · PROJECTS · REVISION**	70
70	INTERVIEW BANK	70
71	PROJECT 1 · REPORT CARD SYSTEM	77
72	PROJECT 2 · RIDEX FARE ENGINE	78
73	PROJECT 3 · LIBRARY LITE	79
74	THE MASTER MAP	80
75	ONE-PAGE CHEAT SHEET	81

✦ THE STORY OF OOP ✦

Every new idea in this notebook exists because the previous idea hit a wall. Read this page first, and come back to it whenever a chapter feels disconnected.

problem → the concept that fixes it

THE PROBLEM WE HIT		THE CONCEPT THAT FIXES IT
1 student's data is spread over 3 arrays	→	CLASS (make your own type)
a class is only a plan, no real student	→	OBJECT (+ new, references)
every object starts empty, set 1 by 1	→	CONSTRUCTOR
field and parameter have the same name	→	this
some data belongs to ALL objects	→	static
two classes called Greeting collide	→	PACKAGES
same code copy-pasted into many classes	→	INHERITANCE
one call should behave differently	→	POLYMORPHISM
anyone can write marks = -50	→	ENCAPSULATION
user drowns in internal details	→	ABSTRACTION
"every child MUST define career()"	→	ABSTRACT CLASS
a class needs MANY parents' contracts	→	INTERFACE
our ArrayList only stores int	→	GENERICS
one bad input crashes the whole program	→	EXCEPTIONS
b = a copies only the arrow	→	CLONING
arrays are fixed size, no ready tools	→	COLLECTIONS
"day" could be any random String	→	ENUMS

the learning path

Class → Object → Constructor → this → static → Packages

↓

Inheritance → Polymorphism → Encapsulation → Abstraction

↓

Abstract class → Interface → Generics → Lambdas → Exceptions

↓

Cloning → Collections → Enums ★ done

Chapter	Lecture	What it covers
01	OOP 1	classes, objects, references, `new`, constructors, `this`, wrappers, `final`, garbage collection
02	OOP 2	packages, `import`, `static`, static blocks, `main()`, inner classes, singleton, `System.out.println` internals
03	OOP 3	inheritance, `super`, polymorphism, overloading, overriding, encapsulation, abstraction
04	OOP 4	access modifiers, built-in packages, the `Object` class, `equals` / `hashCode`, `instanceof`
05	OOP 5	abstract classes, interfaces, default/static methods, nested interfaces, annotations
06	OOP 6	custom ArrayList, generics, wildcards, Comparable/Comparator, lambdas, exceptions, cloning
07	OOP 7	collections framework, `Vector` & synchronization, enums
★	—	interview bank (85 Q), 3 mini projects, master map, one-page cheat sheet

01 INTRODUCTION TO OOP

classes · objects · references · new · constructors · this · wrapper classes · final · garbage collection

✦ WHY OOP? ✦

🧠 IDEA

The real world is made of **things** — students, cars, bank accounts. Each thing has data (name, balance) and does work (greet, withdraw). Object-Oriented Programming lets code copy that shape: bundle data + behaviour into one unit called an **object**.

❓ WHY DO WE NEED IT? — feel the problem first

Store roll no, name and marks of 5 students:

😩 without OOP

```
int[]    rno   = new int[5];
String[] name  = new String[5];
float[]  marks = new float[5];
// student #3 = rno[2] + name[2] + marks[2]  ← scattered in 3 arrays!
```

- one student's data lives in **3 different places**
- add "phone no." → change every array and every loop
- nothing stops index mix-ups between arrays
- where does `greeting()` belong? to whom?

😍 with OOP

```
class Student {              // ONE new data type that groups everything
    int rno;
    String name;
    float marks;
}

Student[] students = new Student[5];    // 5 students, ONE array
```

💡 CORE IDEA

A class lets you **create your own data type**. `int` stores one number — `Student` stores a whole student.

🌍 IN REAL LIFE

*Procedural code = **a recipe**: step 1, step 2, step 3. OOP = **a restaurant**: chef, waiter, cashier — each one knows its own data and its own job, and they work by talking to each other.*

	PROCEDURAL	OBJECT-ORIENTED
Thinks in	steps / functions	objects (data + behaviour)
Data	separate, often global	inside objects, can be hidden
Reuse	copy functions around	inheritance, composition
Safety	anyone can change any data	access control (`private` ...)
Large projects	gets tangled fast	maps the real world, scales
Examples	C	Java, C#, C++ (C++ & Python are multi-paradigm)

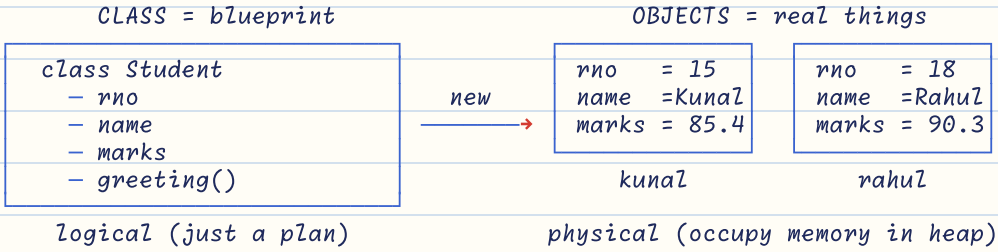
☕ JAVA REALITY

Java is not "100 % pure" OOP — primitives (`int`, `char`, ...) are not objects and `static` members exist without any object. It's a deliberate speed/convenience trade-off.

✦ CLASS & OBJECT ✦

🧠 IDEA

`Class` = **blueprint / template**. `Object` = the **actual thing** built from it — an instance of the class.



🌍 IN REAL LIFE

An architect's **house plan** → you can't sleep in a plan, but you can build 100 houses from it. Every house follows the same plan, yet each can have its own paint colour (its own state).

✧ *finding classes in a problem:*
nouns → classes
verbs → methods

📖 DEFINITION

- `Class` — a named group of **properties** (fields) and **functions** (methods). It creates a new data type.
- `Object` — an instance of a class. Exists at runtime, occupies memory.
- `Instance variable (field)` — declared inside the class, outside methods. Every object gets its own copy.
- `Method` — a function inside a class; the object's behaviour.

✍️ SYNTAX

Java

```
class ClassName {
    // fields → state
    // methods → behaviour
}

ClassName ref = new ClassName();    // create an object
ref.field;                          // dot operator → reach into that object
ref.method();
```

💻 EXAMPLE 1

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Student kunal = new Student();
        kunal.rno = 15;
        kunal.name = "Kunal";
        kunal.greeting();

        Student rahul = new Student();        // never touched
        System.out.println(rahul.name + " " + rahul.rno
                           + " " + rahul.marks);
    }
}

class Student {
    int rno;
    String name;
    float marks = 90;       // explicit initial value

    void greeting() {
        System.out.println("Hello! My name is " + name);
    }
}
```

▶ Output

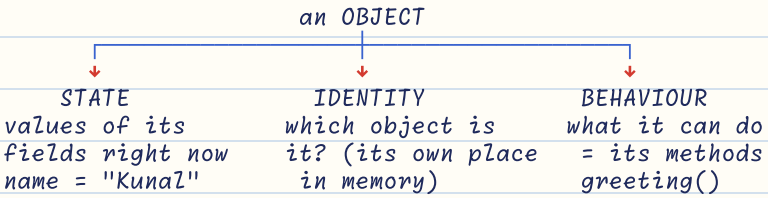
```
Hello! My name is Kunal
null 0 90.0
```

🔎 WHAT JUST HAPPENED?

1) `new Student()` → a fresh object in the heap; every field gets a **default value** (unless initialised, like `marks = 90`).
2) `kunal.rno = 15` → the dot operator reaches kunal's copy of `rno`.
3) `rahul` is a different object → `null 0 90.0`.

field type	default value
`byte short int long`	`0`
`float double`	`0.0`
`char`	`'\u0000'`
`boolean`	`false`
any reference (`String`, arrays, objects)	`null`

🧠 VISUALIZE IT — 3 properties of every object



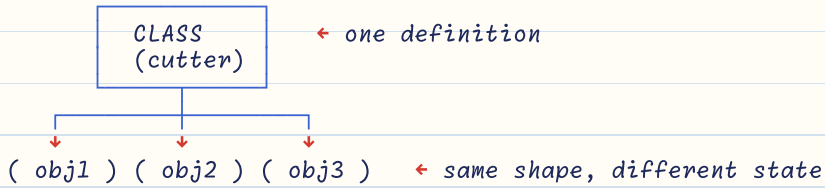
⚠️ COMMON MISTAKE

`Student kunal;` creates **no object** — only an empty reference variable. Use a local one before assigning → × compile error. A field left like that is `null` → `NullPointerException` when you use it.

✦ CLASS vs OBJECT ✦

	CLASS	OBJECT
What	*blueprint / template*	*instance / real thing*
Nature	*logical construct*	*physical — occupies heap memory*
Created with	`class` *keyword, written once*	`new`*, as many times as you like*
Memory	*class info loaded once by JVM*	*each object → its own copy of fields*
Example	`class Car { }`	`new Car()`
Analogy	*cookie cutter*	*cookies*

one cutter, many cookies



⭐ REMEMBER

- *A class is written once; objects are created at* ***runtime*** *with* `new`.
- *Two objects of the same class are independent — changing one doesn't change the other.*
- *Methods are shared code;* ***fields*** *are per-object (until we meet* `static`*).*

✦ REFERENCES, new & MEMORY ✦

IDEA

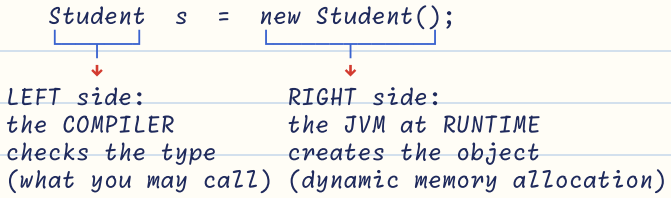
`new` ***creates the object in the heap*** *at runtime and* ***returns a reference*** *to it — think of it as an arrow / address-like handle. The variable stores only that arrow,* ***never the object itself****.*

Java

```
Student s;          //① declare a reference — no object yet
s = new Student();  //② create object in heap, store its reference in s
```



Who looks at which side?



Two references, ONE object

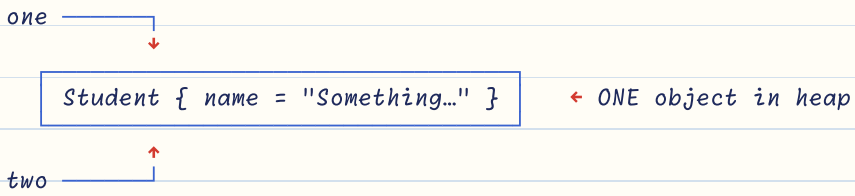
Main.java

```
public class Main {
    public static void main(String[] args) {
        Student one = new Student();
        Student two = one;          // copies the ARROW, not the object
        one.name = "Something something";
        System.out.println(two.name);
    }
}

class Student { String name; }
```

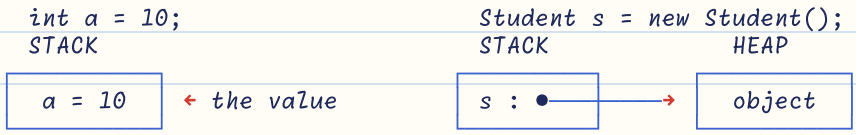
▶ *Output*

```
Something something
```



❓ Why no `new` for `int`?

*Primitives are **not objects** — the variable holds the value itself. No heap trip → fast.*



☕ JAVA REALITY

*"Locals on the stack, objects on the heap" is the right mental model. Fields — even `int` fields — live inside their object on the heap, and the JIT may optimise allocations away. A Java reference is **not** a pointer you can do maths on — no pointer arithmetic is part of what makes Java safe.*

🎯 INTERVIEW TRAP #1

`Student b = a;` *does **not** copy the object — it copies the **reference**. Both names now control the same object. (Real copying = cloning, Ch 06.)*

✦ REFERENCE vs OBJECT ✦

🌍 IN REAL LIFE — remote & TV

The **remote** is the reference; the **TV** is the object. Two remotes can control one TV. A remote can be paired to nothing (`null`). Lose every remote → nobody can reach that TV → it's garbage.

	REFERENCE	OBJECT
Is	an arrow / handle	the actual block of data
Lives	stack (local var) or inside another object (field)	heap
Made by	a declaration: `Student s;`	`new Student()`
Can be `null`?	✓ yes	× an object is never "null"
Re-pointable?	✓ (unless `final`)	—
`==` on two refs	compares arrows: same object?	—
Its type decides	what you're **allowed to call** (compiler)	which overridden method **runs** (JVM, Ch 03)

the 4 words students mix up

```
VARIABLE   = the named box          →  s
REFERENCE  = what the box holds     →  ●  (an arrow)
OBJECT     = what the arrow points to →  Student { … }
MEMORY     = where they live        →  s on STACK · object on HEAP
```

⚠️ COMMON MISTAKE

Saying "`s` **is** a Student object". Better: "`s` **refers to** a Student object." Getting this sentence right fixes half of all OOP confusion.

✦ CONSTRUCTORS ✦

❓ WHY?

Filling fields one by one (`kunal.rno = 15; kunal.name = …`) is boring and easy to forget. We want to ***create + initialise in one line***:

Java

```java
Student kunal = new Student(15, "Kunal Kushwaha", 85.4f);
```

🌍 IN REAL LIFE

When a house is built, electricity, paint and keys are set up ***exactly once***, at construction time — before anyone moves in.

📖 DEFINITION

A ***constructor*** is a special block that runs automatically when an object is created with `new`, before `new` hands back the reference. It has the ***same name as the class*** and ***no return type*** — not even `void`.

⭐ JAVA RULE

- No constructor written → compiler adds a ***default no-arg constructor***.
- You write ***any*** constructor → the default one is ***not*** added.
- `void Student() { }` is ***not*** a constructor — it's an ordinary method that happens to share the class name!

💻 EXAMPLE — all 3 kinds (the class example)

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Student kunal  = new Student(15, "Kunal Kushwaha", 85.4f);
        Student random = new Student(kunal);    // copy-style
        Student def    = new Student();         // no-arg
        System.out.println(kunal.name + " | " + random.name);
        System.out.println(def.name + " " + def.marks);
    }
}

class Student {
    int rno; String name; float marks = 90;

    Student() {                                          //① no-arg
        this(13, "default person", 100.0f);              //  → delegates to ③
    }
    Student(Student other) {                             //② takes another object
        this.rno = other.rno;
        this.name = other.name;
        this.marks = other.marks;
    }
    Student(int rno, String name, float marks) {    //③ parameterized
        this.rno = rno; this.name = name; this.marks = marks;
    }
}
```

▶ Output

```
Kunal Kushwaha | Kunal Kushwaha
default person 100.0
```

Three constructors, same name, different parameters → constructor overloading. Java picks the one whose parameters match the arguments.

⚠️ The disappearing default constructor

Box.java

```java
class Box {
    int l;
    Box(int l) { this.l = l; }
}

class Test {
    void make() {
        Box b = new Box();      //× no Box() exists any more
    }
}
```

💥 *compile error*

```
error: constructor Box in class Box cannot be applied to given types;
  required: int
  found:    no arguments
```

	CONSTRUCTOR	METHOD
Name	same as class	anything
Return type	none (not even `void`)	must have one (`void` counts)
Called	automatically by `new`, once per object	explicitly, any number of times
Inherited?	× never	✓ (non-private)
Can be `static / final / abstract`?	×	✓

🎯 INTERVIEW TRAP #2

"Constructors return nothing, so `new` returns nothing?" × — the constructor has no return type, but the `new` **expression** evaluates to the reference of the new object.

✦ THE `this` KEYWORD ✦

🧠 IDEA

`this` = a reference to the ***current object*** — the object on which the method or constructor is running right now.

the classic use

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
             ↑      ↑
             |      └── the PARAMETER (local variable)
             └── the FIELD of the current object
    }
}
```

⚠️ COMMON MISTAKE

`name = name;` assigns the parameter ***to itself*** (the local variable *shadows* the field). The field stays `null`. Use `this.name = name;`.

🧠 VISUALIZE IT — `this` changes with the caller

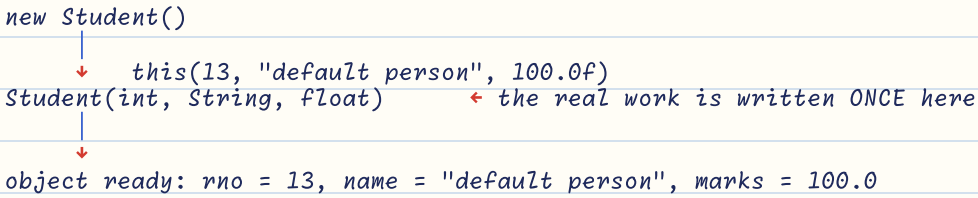
```
kunal.greeting();   →   inside greeting():  this == kunal
rahul.greeting();   →   inside greeting():  this == rahul

kunal ──→ [ name = Kunal ] ←── this
                               (while kunal.greeting() runs)
```

✍️ 4 uses of `this`

1) `this.field` — separate a field from a same-named parameter
2) `this(...)` — call another constructor of the ***same*** class (constructor chaining)
3) `obj.compare(this)` — pass the current object as an argument
4) `return this;` — return the current object → calls can be chained: `acc.deposit(10).deposit(20)`

💻 CONSTRUCTOR CHAINING



⭐ JAVA RULE

`this(...)` (and `super(...)`, Ch 03) must be the ***first statement*** of a constructor — and a constructor can't call both.

☕ JAVA REALITY

Since **Java 25** ("flexible constructor bodies") you may write statements before `this(...)` / `super(...)` — e.g. argument checks — as long as they don't use the object being built. Exams and most codebases still follow the classic "first statement" rule.

🎯 INTERVIEW TRAP #3

`this` cannot appear in a `static` method — a static method runs without any current object (Ch 02).

✦ WRAPPER CLASSES ✦

🧠 IDEA

Every primitive has an **object version** in `java.lang`:

primitive	byte	short	int	long	float	double	char	boolean
wrapper	Byte	Short	Integer	Long	Float	Double	Character	Boolean

❓ WHY?

- collections & generics hold **objects only** → `ArrayList<int>` ✗ · `ArrayList<Integer>` ✓
- a wrapper can be `null` ("no value yet")
- handy helpers: `Integer.parseInt("42")`, `Integer.MAX_VALUE`, `Character.isDigit(c)`

Java

```
Integer num = 45;    // autoboxing: int → Integer  (Integer.valueOf(45))
int n = num;         // unboxing:   Integer → int  (num.intValue())
```

💻 The swap puzzle

WrapperExample.java

```
public class WrapperExample {
    public static void main(String[] args) {
        Integer a = 10, b = 20;
        swap(a, b);
        System.out.println(a + " " + b);
    }

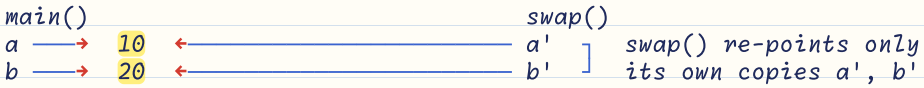
    static void swap(Integer a, Integer b) {
        Integer temp = a;
        a = b;
        b = temp;
    }
}
```

▶ Output

```
10 20
```

🔎 WHAT JUST HAPPENED?

Java is **always pass-by-value**. For objects, the value that gets copied is the reference. `swap()` shuffles **its own copies** of the arrows; `main`'s `a` and `b` never move. (And `Integer` is immutable, so the 10-object can't turn into 20 either.)



Integer cache #4

`Integer x = 127, y = 127;` → `x == y` is **true** (values −128…127 are cached).

`Integer p = 128, q = 128;` → `p == q` is **false** (two different objects). Compare wrappers with `.equals()`, never `==`.

⚠ COMMON MISTAKE

`Integer v = null; int k = v;` → unboxing `null` throws **NullPointerException**.

✦ THE `final` KEYWORD ✦

🧠 IDEA

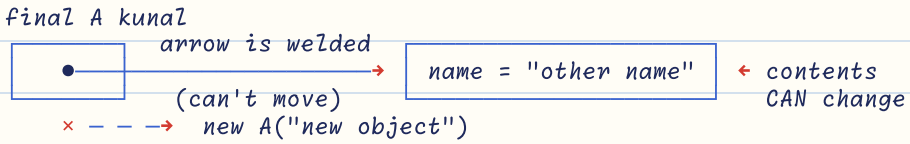
`final` = ***assign once, never re-assign.***

- *final variable → a constant; must be initialised exactly once (at declaration, or — for a field — in every constructor)*
- *convention:* `static final int MAX_SIZE = 10;` → *UPPER_SNAKE_CASE*
- *final method → can't be overridden · final class → can't be extended (both in Ch 03)*

final with primitives vs references

```
final int bonus = 2;
bonus = 3;                          // ✗ compile error

final A kunal = new A("Kunal Kushwaha");
kunal.name = "other name";          // ✓ allowed — the OBJECT changes
kunal = new A("new object");        // ✗ compile error — the ARROW is fixed
```



🎯 INTERVIEW TRAP #5

A `final` *reference ≠ an immutable object.* `final` *freezes the* ***arrow****, not the* ***object****. Immutability (like* `String`*) must be designed into the class: private final fields, no setters.*

?

"Must a final field be given a value right away?"

No — a ***blank final*** *may be assigned later, but exactly once on every path of every constructor:*

`final int VALUE; Parent(int age) { VALUE = 32456789; }` ✓

✦ GARBAGE COLLECTION ✦

🧠 IDEA

An object that **no reference can reach** is useless. The JVM's **Garbage Collector (GC)** finds such objects and frees their memory automatically — no `free()` / `delete` like C/C++.

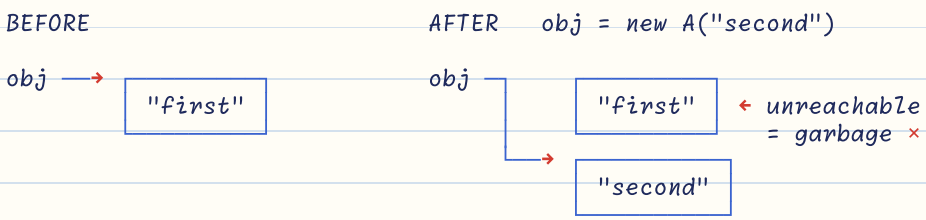
🌍 IN REAL LIFE

Food-court trays: once you walk away from your table, staff clear the tray. You don't clean it yourself — and you don't decide when they come.

✧ you can't delete objects in Java — you just let go of them

Java

```
A obj = new A("first");
obj = new A("second");     // "first" has no reference left → eligible for GC
```



4 ways an object becomes eligible

1) its reference is re-assigned (above)
2) its reference is set to `null`
3) the method holding the only reference finishes (local goes out of scope)
4) "island of isolation" — objects pointing only at each other, nothing outside points at them

☕ JAVA REALITY

- `System.gc()` is only a **request**; the JVM may ignore it.
- Old code overrides `finalize()` (e.g. to print "Object is destroyed"). It is **deprecated** (since Java 9, for removal since Java 18): it may run late or never. For cleanup use `close()` / try-with-resources.

🎯 INTERVIEW TRAP #6

"GC destroys an object the moment it becomes unreachable" × — it only becomes eligible. When (or whether) GC runs is the JVM's decision.

⭐ CHAPTER 01 – REMEMBER

⭐ REMEMBER

- class = blueprint (logical); object = instance (physical, heap)
- `new` allocates at runtime and returns a **reference**; `b = a` copies only the arrow
- constructor: class name, no return type; the default one exists only if you write none
- `this` = current object; `this(...)` goes first in a constructor
- `final` reference ≠ immutable object · unreachable ≠ destroyed (just eligible)

🧪 PRACTICE – CHAPTER 01

🧪 QUICK CHECK

1) `Student a = new Student(); Student b = a; b.name = "X";` → what is `a.name`?
2) A class has only `Box(int side)`. Does `new Box()` compile? Why?
3) Inside a constructor you wrote `name = name;` — what is the field's value afterwards?

💻 CODE PRACTICE

1) `BankAccount` with `accountNo`, `holder`, `balance`: three constructors chained with `this(...)`, and `deposit(double)` that `return this;` so `acc.deposit(100).deposit(50)` works.
2) Create 3 `Student` objects, then make two of them unreachable in two different ways. Draw the stack/heap picture after every line.

🎯 INTERVIEW THINKING

`final int[] arr = {1, 2, 3}; arr[0] = 100;` — does it compile? What does `arr[0]` hold? And `arr = new int[3];`?

☑ Answers

1) `"X"` — one object, two arrows. 2. × — writing any constructor removes the default one. 3. The default (`null`) — the parameter was assigned to itself. **Interview:** ✓ compiles and `arr[0]` is `100` — the array object can change. `arr = new int[3];` × — the final arrow can't move.

👨‍🏫 HOW I WOULD TEACH THIS

1) Start with 5 students in 3 arrays — let them *feel* the mess
2) Analogy: house plan → houses
3) Write `class Student`, create two objects
4) Draw stack & heap for **every** `new`
5) Ask: `b = a; b.name = "X";` → `a.name`?
6) Then constructors + `this` to kill the boring assignments

🔗 NEXT →

Every `Student` has its **own** `name`. But some data belongs to **all** students together — like the total population. Who owns that? → `static` (Chapter 02).

02 PACKAGES · STATIC · INNER CLASSES · SINGLETON

package · import · static members & blocks · why main() is static · System.out.println · inner classes · singleton

✦ PACKAGES ✦

🧠 IDEA

A **package** is a folder + a namespace for classes. It groups related classes and lets two classes share a name without clashing.

🌍 IN REAL LIFE

Two students called Rahul in one school → "Rahul of 10-A" and "Rahul of 12-C". The section is the package. Same idea as two files called `notes.txt` in different folders.

❓ WHY DO WE NEED IT?

- our example has **two** classes named `Greeting` — one in package `a`, one in `b` — no clash
- big projects stay organised (`model`, `service`, `util` ...)
- a package is also a **visibility boundary** (default access → Ch 04)

📖 DEFINITION

A `package` is a namespace that groups related classes and interfaces. It is both a **naming** and a **visibility-control** mechanism. The folder structure must mirror the package name.

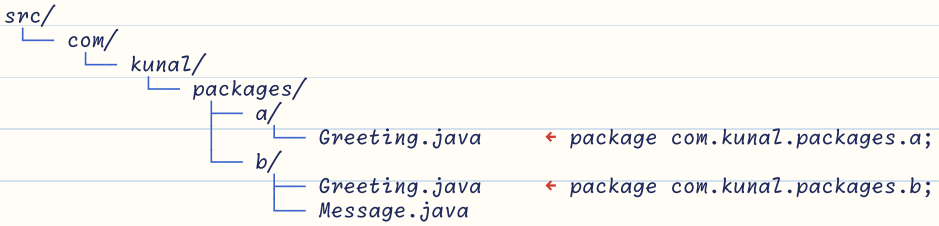
✍️ SYNTAX

src/com/kunal/packages/a/Greeting.java

```
package com.kunal.packages.a;                                  //① FIRST statement

import static com.kunal.packages.b.Message.message;            //② imports

public class Greeting {                                        //③ the class
    public static void main(String[] args) {
        System.out.println("Hello world");
        message();                          // static import → no class name
    }
}
```



📦 import — 4 flavours

- `import java.util.ArrayList;` → one class
- `import java.util.*;` → every class of `java.util` (but **not** its sub-packages)
- `import static java.lang.Math.*;` → static members: `sqrt(16)` instead of `Math.sqrt(16)`
- no import at all → write the **fully qualified name**:

```
com.kunal.packages.b.Greeting g = new com.kunal.packages.b.Greeting();
```

`java.lang` (String, System, Math, Integer, Object …) is imported **automatically**.

⭐ JAVA RULE

- at most **one** `package` statement, and it comes first
- at most **one** `public` **top-level class** per file — and the file must be named after it
- order in a file: `package` → `import`s → classes

🎯 INTERVIEW TRAP #7

`import java.util.*;` does **not** give you `java.util.function.Consumer`. A sub-package is a completely separate package — the dots only look like a hierarchy.

☕ JAVA REALITY

Same simple name from two packages (`java.util.Date` and `java.sql.Date`)? Import one, write the other's fully qualified name. The JVM finds packages via the current directory, the `CLASSPATH` variable, or `-classpath` / `-cp`.

✦ STATIC ✦

❓ WHY? — the problem

`Human` has `age`, `name`, `salary` … and **population**. Age belongs to one person. Population belongs to all humans together. As an instance field, every object would hold its own copy: kunal says 1, rahul says 1, arpit says 1 ×.

🧠 IDEA

A `static` member belongs to the class, not to any object. One copy, shared by everyone, usable even before any object exists.

🌍 IN REAL LIFE

✧ static ≈ ours
instance ≈ mine

A classroom: every student has their **own notebook** (instance), but there is **one whiteboard** (static). Write on the whiteboard → everyone sees the change.

💻 EXAMPLE

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Human kunal = new Human(22, "Kunal");
        Human rahul = new Human(34, "Rahul");
        Human arpit = new Human(34, "Arpit");

        System.out.println(Human.population);    // ClassName.member ✓
    }
}

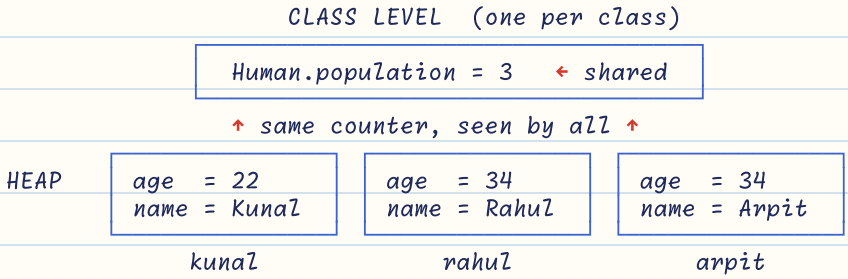
class Human {
    int age;
    String name;
    static long population;              // ONE copy for the whole class

    Human(int age, String name) {
        this.age = age;
        this.name = name;
        Human.population += 1;       // every object bumps ONE counter
    }
}
```

▶ Output

```
3
```

🧠 VISUALIZE IT



✍️ Static methods

A method that doesn't need any object's data can be `static` → call it with the class name: `Math.sqrt(25)`, `Arrays.sort(arr)`, `Integer.parseInt("7")`, `Human.message()`.

⚠️ COMMON MISTAKE

`kunal.population` compiles, but reads like "kunal's own population". Always write `Human.population` — it tells the reader the truth: it's shared.

✦ STATIC — WHO CAN TALK TO WHOM? ✦

FROM ↓ · TO →	static member	instance member
static method	✓ directly	× directly · ✓ through an object reference
instance method	✓ directly	✓ directly

❓ WHY can't static touch instance members directly?

An instance member belongs to **some object**. A static method runs with **no object at all** — there is no `this`. So "`greeting()`" inside a static method means greeting of … whom?

Main.java

```java
public class Main {
    static void fun() {
        // greeting();      × no object here — whose greeting?
        Main obj = new Main();
        obj.greeting();         //✓ now we NAME the object
    }

    void fun2() {
        greeting();             //✓ means this.greeting()
    }

    void greeting() {
        System.out.println("Hello world");
    }
}
```

this inside static

```java
class Human {
    int age;

    static void message() {
        System.out.println(this.age);
    }
}
```

💥 compile error

```
error: non-static variable this cannot be referenced from a static context
```

⭐ JAVA RULE

- static methods: no `this`, no `super`
- static members are created when the class **loads**; instance members when an object is **created**
- local variables can't be `static` → `void f() { static int x; }` ×

☕ JAVA REALITY

"A static method can only call static methods" is the short version. Precisely: it can't call instance methods **directly** (no implicit `this`) — through a reference like `obj.greeting()` it's perfectly fine.

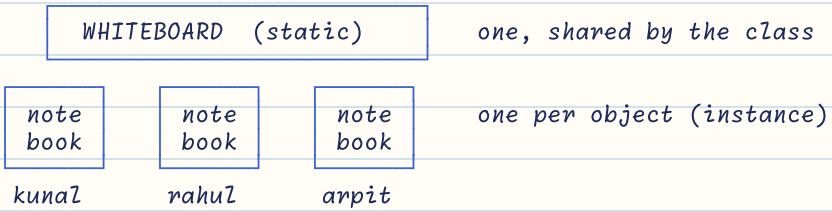
🎯 INTERVIEW TRAP #8

Static methods are **not overridden** — a child's static method with the same signature only *hides* the parent's (Ch 03).

✦ STATIC vs NON-STATIC ✦

	STATIC	NON-STATIC (instance)
Belongs to	the class	each object
Copies	**one**, shared	one **per object**
Access with	`ClassName.member`	`objectRef.member`
Comes alive	when the class is loaded	when an object is created with `new`
`this` / `super`	× not available	✓ available
Can use directly	only static members	static **and** instance members
Overriding	× (hiding only)	✓
Examples	`Human.population`, `Math.PI`, `main()`	`kunal.name`, `kunal.greeting()`

whiteboard vs notebooks



⭐ REMEMBER

- `static` = class-level, shared · instance = object-level, private copy
- a static context has **no** `this` → name an object to reach instance members
- prefer `ClassName.staticMember`, never `object.staticMember`

✦ STATIC BLOCK & INITIALIZATION ORDER ✦

IDEA

A `static { … }` block runs **exactly once** — when the class is first loaded. Use it when a static variable needs computation to initialise.

StaticBlock.java

```java
public class StaticBlock {
    static int a = 4;
    static int b;

    static {                          // runs ONCE, at class loading
        System.out.println("I am in static block");
        b = a * 5;
    }

    public static void main(String[] args) {
        StaticBlock obj = new StaticBlock();
        System.out.println(StaticBlock.a + " " + StaticBlock.b);
        StaticBlock.b += 3;
        System.out.println(StaticBlock.a + " " + StaticBlock.b);
        StaticBlock obj2 = new StaticBlock();  // no static block again
        System.out.println(StaticBlock.a + " " + StaticBlock.b);
    }
}
```

▶ Output

```
I am in static block
4 20
4 23
4 23
```

VISUALIZE IT — the full order

```
class used for the first time  →  JVM loads it
        ↓
① static variables + static { } blocks      top → bottom, ONCE
        ↓
② main() starts
        ↓
        new Order()
③ parent constructor first (super)          every object
        ↓
④ instance variables + instance { } blocks  top → bottom
        ↓
⑤ rest of the constructor body
```

Order.java

```java
public class Order {
    static { System.out.println("1. static block"); }
    { System.out.println("3. instance block"); }
    Order() { System.out.println("4. constructor"); }

    public static void main(String[] args) {
        System.out.println("2. main starts");
        new Order();
        new Order();
    }
}
```

▶ Output

```
1. static block
2. main starts
3. instance block
4. constructor
3. instance block
4. constructor
```

🎯 INTERVIEW TRAP #9

Creating 5 objects runs the static block **once**, but the instance block **5 times**. And the static block of the class holding `main` runs **before** `main` itself.

✦ WHY IS `main()` STATIC? ✦

Java

```
public static void main(String[] args)
```

public	static	void	main	(String[] args)
↓	↓	↓	↓	↓
the JVM (outside the class) can reach it	the JVM calls it WITHOUT creating an object of Main	returns nothing	the exact name the JVM looks for	command-line arguments java Main hi 5 → {"hi", "5"}

? WHY static?

When the program starts **no objects exist yet**. The JVM loads `Main` and needs a door to walk in through. If `main` were an instance method, the JVM would first have to build a `Main` object — with which constructor? which arguments? `static` removes that chicken-and-egg problem: **call it on the class itself**.

public void main(...)

```
public class Main {
    public void main(String[] args) {          // no static
        System.out.println("hi");
    }
}
```

> 💥 classic Java (8 … 24)
>
> ```
> Error: Main method is not static in class Main, please define the main method as:
>    public static void main(String[] args)
> ```

☕ JAVA REALITY

Java 25 (JEP 512) relaxed the launcher: it also accepts an instance `main` (even without `String[] args`) and creates the object for you with the no-arg constructor. The interview answer is still: "main is static so the JVM can call it without creating an object."

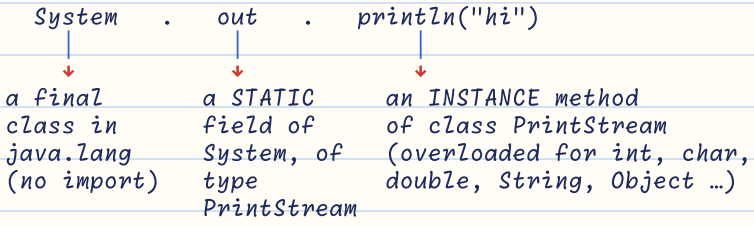
variation	works?
`static public void main(String[] args)`	✓ modifier order doesn't matter
`public static void main(String... args)`	✓ varargs is fine
`public static void main(String args[])`	✓ C-style array
`private static void main(String[] args)`	× "Main method not found"
overloading `main(int x)` **too**	✓ compiles — but the JVM only calls the `String[]` one

⚠️ COMMON MISTAKE

`args` holds **Strings** only: `java Main 5` gives `args[0] = "5"`. Need a number → `Integer.parseInt(args[0])`.

✦ HOW System.out.println() WORKS ✦

GOOD TO KNOW



So `System.out` is a ***static field*** (no `System` object needed), and `println` runs ***on the*** `PrintStream` ***object*** that `out` refers to. Everything from this chapter, in one line!

🖨️ Printing an object

`println(Object x)` → `String.valueOf(x)` → `x.toString()`. Not overridden? You get `ClassName@hexHash`, e.g. `Test@1b6d3586`. Override `toString()` to print something useful:

Main.java

```
public class Main {
    static class Test {
        String name;
        Test(String name) { this.name = name; }

        @Override
        public String toString() {       // without this → Main$Test@<hash>
            return name;
        }
    }

    public static void main(String[] args) {
        Test a = new Test("Kunal");
        System.out.println(a);           // calls a.toString() for you
    }
}
```

▶ Output

```
Kunal
```

🎯 INTERVIEW TRAP #10

`System.out.println(null);` does **not** print "null" — it doesn't even compile: `println(char[])` and `println(String)` both accept `null` → "reference to println is ambiguous".
(`String s = null; System.out.println(s);` prints `null` fine.)

✦ INNER CLASSES ✦

🧠 IDEA

A class declared **inside** another class. Two kinds matter:

- **inner class** (non-static) → every inner object is **tied to one outer object**
- **static nested class** → just filed inside the outer class; needs **no** outer object

🌍 IN REAL LIFE

Inner class = a **room**: it can't exist without its house, and it sees everything in that house. Static nested class = a **spare blueprint kept in the house's folder** — filed under "House", but you can use it with no house built at all.

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();       // needs an Outer object
        inner.show();

        Outer.Nested nested = new Outer.Nested();   // no Outer needed
        nested.show();
    }
}

class Outer {
    private int x = 10;
    static int y = 20;

    class Inner {                       // non-static → tied to one Outer
        void show() {
            System.out.println("Inner sees x = " + x + ", y = " + y);
        }
    }

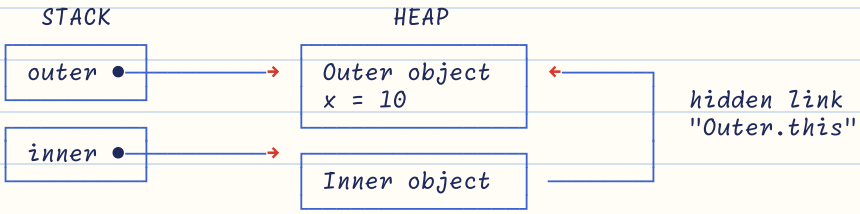
    static class Nested {               // static → belongs to the class Outer
        void show() {
            System.out.println("Nested sees y = " + y);
        }
        // using x here ✗ — which Outer's x? there is no Outer object
    }
}
```

▶ Output

```
Inner sees x = 10, y = 20
Nested sees y = 20
```

🧠 VISUALIZE IT — execution sketch

```
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();
```




```
Outer.Nested n = new Outer.Nested();      ← no link, no Outer needed
```

❓ WHY do the two syntaxes differ?

- `outer.new Inner()` → an inner object is built **for a particular outer object** and silently stores a link to it (`Outer.this`). So you must say *which* outer: `outer.`
- `new Outer.Nested()` → `Nested` is really an independent class with a long name; `Outer.` is only its address.

puzzle — make Test non-static

```java
public class InnerClasses {
    class Test {                        // static removed
        String name;
        Test(String name) { this.name = name; }
    }

    public static void main(String[] args) {
        Test a = new Test("Kunal");     // main has no InnerClasses object!
    }
}
```

> 💥 *compile error*
>
> `error: non-static variable this cannot be referenced from a static context`

⭐ JAVA RULE

- only **nested** classes can be `static` — `static class A { }` at top level ×
- an inner class can read the outer class's **private** members (and vice-versa)

☕ JAVA REALITY

Since Java 16 an inner (non-static) class may also declare `static` members — older notes say only static nested classes can. Two more kinds you'll meet later: **local** classes (inside a method) and **anonymous** classes (Comparator in Ch 06).

✦ SINGLETON CLASS ✦

❓ WHY?

*Some things must exist **only once**: app configuration, a logger, a connection pool. Many copies → wasted resources and conflicting state.*

🌍 IN REAL LIFE

*A country has **one** President at a time. Everyone asking "who is the President?" gets the **same** person.*

🧠 IDEA — 3 ingredients

1) `private` constructor → nobody outside can write `new`
2) a `private static` field holding **the** instance
3) a `public static` method → the only door in

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Singleton obj1 = Singleton.getInstance();
        Singleton obj2 = Singleton.getInstance();
        Singleton obj3 = Singleton.getInstance();
        System.out.println(obj1 == obj2 && obj2 == obj3);
        // new Singleton(); ✕ Singleton() has private access
    }
}

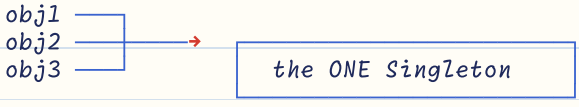
class Singleton {
    private static Singleton instance;      // the ONE object

    private Singleton() { }                 //① no outside new

    public static Singleton getInstance() { //② the only door
        if (instance == null) {             //③ first call only (lazy)
            instance = new Singleton();
        }
        return instance;
    }
}
```

▶ Output

```
true
```



❓ *"Why must getInstance() be static?"*

*Because before the first call **no object exists** — and you can't call an instance method without an object. Static is the only way in.*

☕ **JAVA REALITY**

This lazy version is not thread-safe — two threads may both see `null` and create two objects. Real code uses an eager `private static final` instance, a `synchronized` /double-checked `getInstance()`, or simply `enum Config { INSTANCE }` (also safe against reflection & serialization).

🎯 **INTERVIEW TRAP #11**

*A class whose **only** constructor is `private` can't be extended by a normal outside class — the child's constructor has no accessible `super()` to call.*

⭐ CHAPTER 02 – REMEMBER

⭐ **REMEMBER**

- *package = folder + namespace; `package` first; `java.lang` auto-imported; `pkg.*` skips sub-packages*
- `static` *= one shared copy owned by the class; access as* `ClassName.member`
- *static context = no* `this` *→ name an object to use instance members*
- *static block: once per class load · instance block: once per object*
- `main` *is static so the JVM can call it without an object*
- *inner:* `outer.new Inner()` *· static nested:* `new Outer.Nested()`
- *singleton = private constructor + private static instance + public static getter*

🧪 **PRACTICE – CHAPTER 02**

🧪 **QUICK CHECK**

1) `class C { static int s; int i; C() { s++; i++; } }` *— after* `new C(); new C(); C c = new C();` *what do* `C.s` *and* `c.i` *print?*
2) *Can a static method use* `this`*? Explain in one line why.*
3) *You create 5 objects of a class with a static block and an instance block. How many times does each run?*

💻 CODE PRACTICE

1) `Ticket` class: each new ticket gets a unique id 1, 2, 3 … from a static counter; add `static int totalTickets()`.
2) A `Config` singleton holding `appName`. Prove with `==` that two calls return one object; then rewrite it eagerly.

🎯 INTERVIEW THINKING

`Human h = null; System.out.println(h.population);` — `population` is static. NullPointerException or not?

☑ Answers

1) `3 1` — `s` is shared; each object has its own `i`. 2. × — a static method runs with no current object. 3. Static block once, instance block 5 times. **Interview:** no NPE — it prints the value. Static access is resolved from the reference's **type** at compile time; the `null` object is never touched.

👨‍🏫 HOW I WOULD TEACH THIS

1) Ask "where should population live?" — put it in each object, show 1, 1, 1
2) Whiteboard vs notebooks
3) `static` field + `ClassName.member`
4) Let a static method try `this` — read the error together
5) Run `StaticBlock`, ask for the output order
6) Finish with Singleton = private constructor + static

🔗 NEXT →

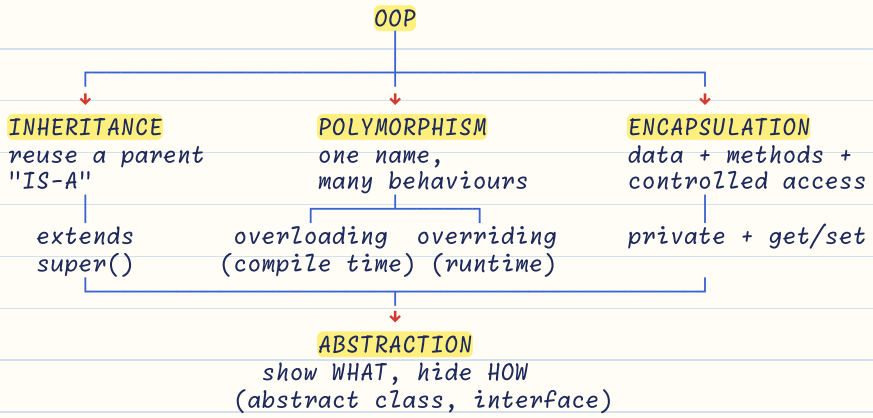
`Box`, `BoxWeight`, `BoxPrice` all need length, width, height. Copy-paste the same fields into three classes? No → **inheritance** (Chapter 03).

03 THE FOUR PILLARS OF OOP

inheritance · super · types of inheritance · polymorphism · overloading · overriding · dynamic dispatch · final · encapsulation · abstraction

✦ THE 4 PILLARS ✦

the map of this chapter



pillar	one line	real life
Inheritance	child reuses & extends the parent's code	a child inherits the family's features
Polymorphism	same call → different behaviour	one person: student, athlete, son, employee
Encapsulation	data hidden, reached only through checked methods	an ATM: you press buttons, never touch the cash box
Abstraction	expose only what's essential	driving without knowing engine internals

💡 CORE IDEA

All four answer one question: **how do we build big programs without drowning?** Reuse (inheritance), flexibility (polymorphism), safety (encapsulation), simplicity (abstraction).

✦ INHERITANCE ✦

❓ WHY? — the problem

We have `Box` with `l, h, w`. Now we need `BoxWeight` (+ weight) and `BoxPrice` (+ cost). Copy-paste `l, h, w` and the constructors into each? Fix a bug → fix it 3 times ×.

🧠 IDEA

A **child class** gets all the (non-private) members of a **parent class** and adds only what's new.
Relationship: **IS-A** — a `BoxWeight` **is a** `Box`.

🌍 IN REAL LIFE

A child inherits common traits from the parents (surname, eye colour) and adds their own (their skills).

📖 DEFINITION

`Inheritance` — a mechanism where one class (**subclass / child / derived**) acquires the properties and behaviour of another (**superclass / parent / base**) using `extends`.

💻 EXAMPLE — the Box family

Main.java

```java
public class Main {
    public static void main(String[] args) {
        BoxWeight box = new BoxWeight(2, 3, 4, 8);
        System.out.println(box.h + " " + box.w + " " + box.weight
                           + " " + box.getL());
    }
}

class Box {
    private double l;            // private → only Box's code can touch
    double h, w;

    Box(double l, double h, double w) {
        System.out.println("Box class constructor");
        this.l = l;
        this.h = h;
        this.w = w;
    }

    public double getL() { return l; }
}

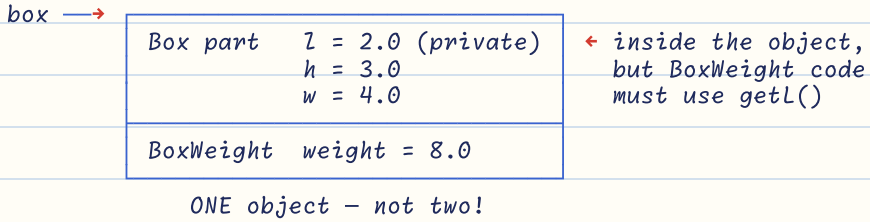
class BoxWeight extends Box {        // BoxWeight IS-A Box
    double weight;                   // only the NEW stuff

    BoxWeight(double l, double h, double w, double weight) {
        super(l, h, w);              // let the parent set up its own part
        this.weight = weight;
    }
}
```

▶ Output

```
Box class constructor
3.0 4.0 8.0 2.0
```

🧠 VISUALIZE IT — one object, two parts



🔑 Parent reference → child object

the experiment

```
Box box5 = new BoxWeight(2, 3, 4, 8);    // ✓ a BoxWeight IS-A Box
box5.w;                                  // ✓ w is declared in Box
box5.weight;                             // ✗ Box has no 'weight'

BoxWeight box6 = new Box(2, 3, 4);       // ✗ compile error
```

❓ *"Why is* `BoxWeight box6 = new Box(…)` *an error?"*

The reference type `BoxWeight` *promises you can use* `box6.weight`*. But a plain* `Box` *object* ***has no weight part*** *— that promise would be a lie. So the compiler refuses. Parent ref → child object ✓; child ref → parent object ✗.*

⭐ JAVA RULE

- *a class can* `extends` ***only one*** *class*
- ***private*** *members are not accessible in the child (they still exist inside the object)*
- ***constructors are never inherited*** *— the child calls them with* `super(...)`

✦ super & THE CONSTRUCTOR CHAIN ✦

🧠 IDEA

`super` = "my **parent's** part". Two forms:

1) `super(...)` → call the **parent's constructor** (must be the first statement)
2) `super.member` → use a parent's field/method that the child has hidden or overridden

💻 Constructors run parent-first

Main.java

```java
public class Main {
    public static void main(String[] args) {
        new BoxPrice(5, 8, 200);
    }
}

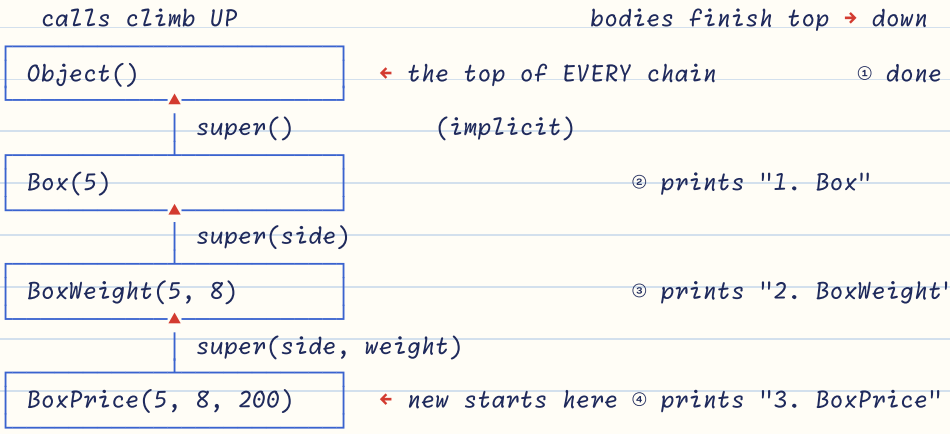
class Box {
    double l, h, w;
    Box(double side) {
        // super(); ← added for you → calls Object()
        l = h = w = side;
        System.out.println("1. Box");
    }
}

class BoxWeight extends Box {
    double weight;
    BoxWeight(double side, double weight) {
        super(side);
        this.weight = weight;
        System.out.println("2. BoxWeight");
    }
}

class BoxPrice extends BoxWeight {
    double cost;
    BoxPrice(double side, double weight, double cost) {
        super(side, weight);
        this.cost = cost;
        System.out.println("3. BoxPrice");
    }
}
```

▶ Output

```
1. Box
2. BoxWeight
3. BoxPrice
```



Why parent first? *The parent knows nothing about the child, but the child may depend on the parent's fields already being ready.*

⚠️ No `super(...)` written?

Java silently inserts `super();` — the parent's **no-arg** constructor. If the parent doesn't have one:

Child.java

```
class Parent {
    Parent(int x) { }          // only a parameterized constructor
}

class Child extends Parent {
    Child() { }                // hidden super() → Parent() doesn't exist
}
```

💥 *compile error*

```
error: constructor Parent in class Parent cannot be applied to given types;
  required: int
  found:    no arguments
```

💻 `super.method()` — reuse, then add

Java

```
class BoxWeight extends Box {
    @Override
    void show() {
        super.show();                                  // parent's version first
        System.out.println("weight = " + weight);      // then the extra part
    }
}
```

🎯 INTERVIEW TRAP #12

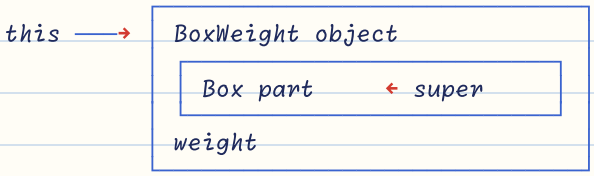
`new BoxPrice(...)` creates **one** object, not three. `super(...)` initialises the parent part of the same object — no separate parent object is ever made.

🎯 INTERVIEW TRAP #13

Constructors are not inherited *and therefore can't be overridden.* `BoxWeight` *doesn't get* `Box(double l, double h, double w)` *for free — it must declare its own and pass values up with* `super(...)`.

this vs super

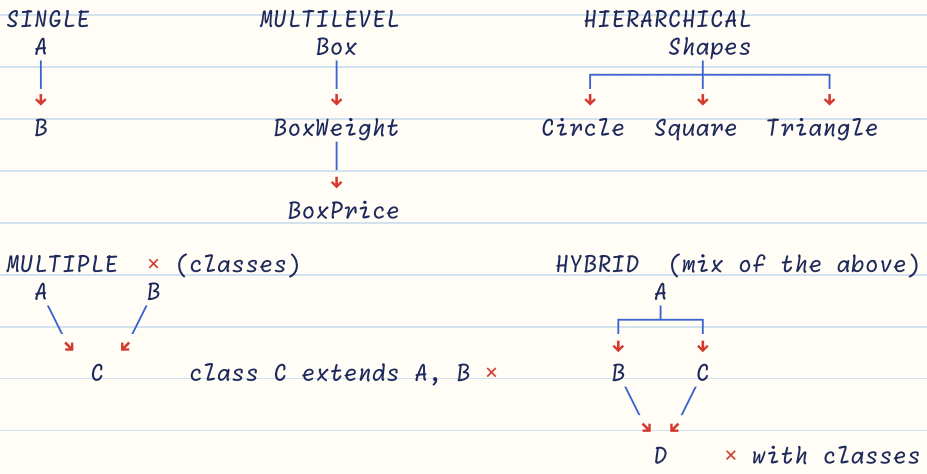
	this	super
Refers to	the current object	the **parent part** of the current object
Field	`this.x` → this class's field	`super.x` → the parent's (hidden) field
Method	`this.f()` → this class's version	`super.f()` → the parent's version
Constructor call	`this(...)` → another constructor of the **same** class	`super(...)` → a constructor of the **parent**
Position	first statement	first statement
Added automatically?	never	`super()` — when you write neither `this(...)` nor `super(...)`
In a static method	×	×



⚠ **COMMON MISTAKE**

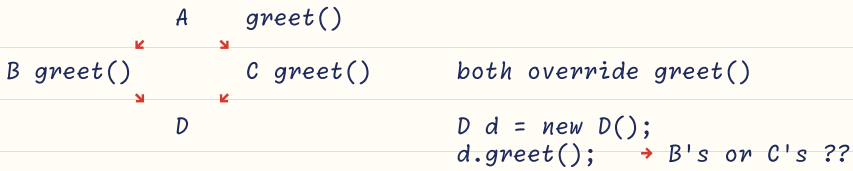
Writing `this(...)` **and** `super(...)` in one constructor × — both want to be first. Put `super(...)` in the constructor that `this(...)` delegates to.

✦ TYPES OF INHERITANCE ✦



type	Java classes?
Single	✓
Multilevel	✓
Hierarchical	✓
Multiple	× — ✓ only through **interfaces** (Ch 05)
Hybrid	× with classes — ✓ through interfaces

💎 Why no multiple inheritance? — the diamond problem



Two parents could provide **different** code for the same method — the child can't pick. Two parents also means two constructor chains and two copies of A's fields. Java avoids the whole mess: **one parent class**, but **many interfaces**.

🎯 INTERVIEW TRAP #14

"Java doesn't support multiple inheritance" — precisely: no multiple inheritance **of classes (state & implementation)**. A class can implement many interfaces (multiple inheritance **of type**).

⚠️ COMMON MISTAKE

`class A extends A` or `A extends B, B extends A` — cyclic inheritance × compile error.

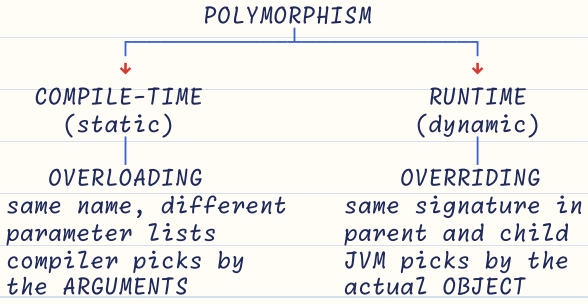
✦ POLYMORPHISM ✦

🧠 IDEA

Poly = many, **morph** = forms → one name, many behaviours.

🌍 IN REAL LIFE

The same person acts as a **student** in class, an **athlete** on the field, a **son** at home, an **employee** at work. One person, many forms — the situation decides the behaviour.



✦ METHOD OVERLOADING ✦ (compile-time)

Same method name, **different parameter list** — in the same class (or inherited).

Numbers.java

```java
public class Numbers {
    int    sum(int a, int b)        { return a + b; }
    double sum(double a, int b)     { return a + b; }
    int    sum(int a, int b, int c) { return a + b + c; }

    public static void main(String[] args) {
        Numbers obj = new Numbers();
        System.out.println(obj.sum(2, 3));       // sum(int, int)
        System.out.println(obj.sum(2.5, 3));     // sum(double, int)
        System.out.println(obj.sum(1, 3, 7));    // 3 params
    }
}
```

▶ Output

```
5
5.5
11
```

⭐ JAVA RULE

Overload by **number**, **type** or **order** of parameters. Return type alone × — Java can't choose by what you do with the result.

Calc.java

```
class Calc {
    int    add(int a, int b) { return a + b; }
    double add(int a, int b) { return a + b; }    // only return type differs
}
```

> 💥 compile error
>
> ```
> error: method add(int,int) is already defined in class Calc
> ```

☕ Automatic type promotion

Overload.java

```
public class Overload {
    static void test(double a) { System.out.println("test(double) " + a); }

    public static void main(String[] args) {
        int i = 88;
        test(i);          // no test(int) → int is widened to double
    }
}
```

> ▶ Output
>
> ```
> test(double) 88.0
> ```

Java widens (`int → long → float → double`) **only if no exact match exists**.

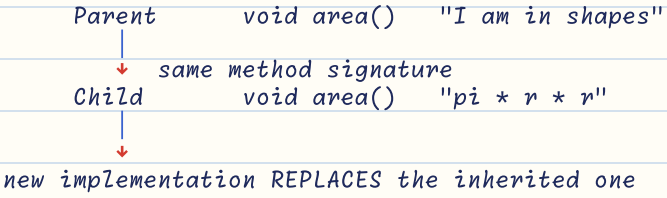
🔗 CONNECT THIS WITH

Constructor overloading (Ch 01) is the same idea. And `System.out.println` is heavily overloaded — `println(int)`, `println(String)`, `println(Object)` …

✦ METHOD OVERRIDING ✦ (runtime)

🌍 IN REAL LIFE

A child inherits "cook dinner" from the parent but **redefines how** — same task name, new way of doing it.



💻 EXAMPLE – Shapes

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Shapes shape  = new Shapes();
        Shapes circle = new Circle();
        Shapes square = new Square();
        shape.area();
        circle.area();
        square.area();
    }
}

class Shapes {
    void area() { System.out.println("I am in shapes"); }
}

class Circle extends Shapes {
    @Override                    // "compiler, check that I really override"
    void area() { System.out.println("Area is pi * r * r"); }
}

class Square extends Shapes {
    @Override
    void area() { System.out.println("Area is square of side"); }
}
```

▶ Output

```
I am in shapes
Area is pi * r * r
Area is square of side
```

⭐ JAVA RULE

1) **same name + same parameter list** (else it's overloading!)
2) return type: same, or a **subtype** (covariant return — `Box copy()` → `BoxWeight copy()` ✓)
3) access: same or **wider** — `public` → `protected` × ("attempting to assign weaker access privileges")
4) can't override: `final` · `static` (hidden) · `private` (not inherited) · constructors
5) may not throw new or broader **checked** exceptions

🧠 How does Java decide which `area()` runs?

```
Shapes circle = new Circle();
circle.area();

COMPILER  →  "does type Shapes have area()?"   ✓ → call allowed
JVM       →  "what is the OBJECT really?"       Circle
          →  run Circle.area()
             (not in Circle? → look in Shapes → then Object)
```

This runtime lookup is **dynamic method dispatch** — how Java implements runtime polymorphism.

⚠️ COMMON MISTAKE

Forgetting `@Override` and typing `void Area()` or `void area(int r)` → you silently created a **new** method; the parent's still runs. `@Override` turns that bug into a compile error.

✦ POLYMORPHISM:

Parent obj = new Child() ✦

MUST KNOW

```
Parent   obj   =   new Child();
  ↓                   ↓
REFERENCE TYPE      OBJECT TYPE
  ↓                   ↓
WHAT members can I    WHICH overridden method
access?               actually runs?
(compiler · early)    (JVM · runtime · late)
```

💻 EXAMPLE 1 — one loop, many behaviours

say it 10 times:
reference → access
object → behaviour

Main.java

```
public class Main {
    public static void main(String[] args) {
        Shapes[] shapes = { new Circle(), new Square(), new Triangle() };
        for (Shapes s : shapes) {
            s.area();                    // ONE line of code, 3 behaviours
        }
    }
}

class Shapes {
    void area() { System.out.println("Shape area"); }
}
class Circle extends Shapes {
    void area() { System.out.println("pi * r * r"); }
}
class Square extends Shapes {
    void area() { System.out.println("side * side"); }
}
class Triangle extends Shapes {
    void area() { System.out.println("0.5 * b * h"); }
}
```

▶ Output

```
pi * r * r
side * side
0.5 * b * h
```

Add a `Hexagon` tomorrow → the loop doesn't change. That's the real power.

💻 EXAMPLE 2 — the reference limits, the object decides

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();                  // ✓ Dog's version runs (object type)
        // a.fetch();       ✗ type Animal has no fetch()

        if (a instanceof Dog) {
            Dog d = (Dog) a;    // downcast → reference now allows it
            d.fetch();
        }
    }
}

class Animal { void sound() { System.out.println("..."); } }

class Dog extends Animal {
    @Override void sound() { System.out.println("Bark"); }
    void fetch() { System.out.println("Fetching the ball"); }
}
```

▶ *Output*

```
Bark
Fetching the ball
```

Wrong downcast → compiles, but crashes: `Animal x = new Animal(); Dog d = (Dog) x;` → ***ClassCastException***. *Check with* `instanceof` *first.*

💻 EXAMPLE 3 — fields & static methods are NOT polymorphic

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Parent obj = new Child();
        System.out.println(obj.name);   // field       → reference type
        obj.instanceHello();            // overridden  → object type
        obj.staticHello();              // static method → reference type
    }
}

class Parent {
    String name = "parent field";
    void instanceHello()      { System.out.println("Parent instance"); }
    static void staticHello() { System.out.println("Parent static"); }
}

class Child extends Parent {
    String name = "child field";                        // HIDES the field
    @Override void instanceHello() { System.out.println("Child instance"); }
    static void staticHello()      { System.out.println("Child static"); }
}
```

▶ *Output*

```
parent field
Child instance
Parent static
```

what you use via `obj`	decided by	when
which members are allowed	reference type	compile time
a field	reference type	compile time
a static method	reference type	compile time
an overridden instance method	**object type**	**runtime**

🎯 INTERVIEW TRAP #15

Static methods are **hidden, not overridden**. `obj.staticHello()` follows the reference type → "Parent static". Putting `@Override` on a static method is a compile error: "static methods cannot be annotated with @Override".

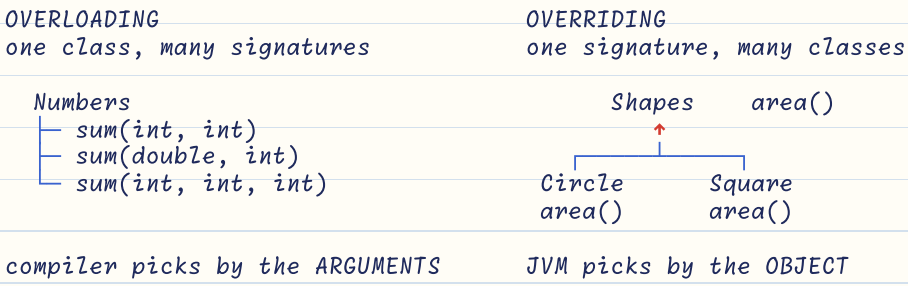
🎯 INTERVIEW TRAP #16

Casting the reference does **not** change the object: `((Parent) new Child()).instanceHello()` still prints "Child instance".

✦ OVERLOADING vs OVERRIDING ✦

MUST KNOW

	OVERLOADING	OVERRIDING
Where	same class (or a child adding a new version)	parent ↔ child — needs inheritance
Name	same	same
Parameters	**must differ** (number / type / order)	**must be identical**
Return type	anything — but it can't be the *only* difference	same or covariant (a subtype)
Access	anything	same or **wider**
`static` **methods**	✓ can be overloaded	× only hidden
`private` / `final` **methods**	✓ can be overloaded	× can't be overridden
Decided by	the **compiler**, from the argument types	the **JVM**, from the object's type
Binding	early (static)	late (dynamic)
Polymorphism	compile-time	runtime
`@Override`	× compile error if used	✓ always write it
Example	`sum(int, int)` · `sum(int, int, int)`	`Circle.area()` replaces `Shapes.area()`



⚠ COMMON MISTAKE

Writing `public boolean equals(Student s)` instead of `equals(Object o)` is **overloading**, not overriding — `HashSet` keeps calling the inherited `equals(Object)`, so your version never runs. `@Override` would have turned this into a compile error.

⭐ REMEMBER

- different parameters → overloading · same signature in a child → overriding
- overloading = compiler decides · overriding = JVM decides at runtime
- changing only the return type or parameter names is neither → "already defined"

✦ final & static IN INHERITANCE ✦

3 jobs of final

on	meaning	example
variable	can't be re-assigned (Ch 01)	`final int MAX = 10;`
method	can't be **overridden**	`final void area() { }`
class	can't be **extended**	`final class String`, all wrapper classes

early vs late binding

normal method → JVM decides at RUNTIME → LATE binding

final / static / private method → can't be overridden → EARLY binding
(compile time — the call may even be inlined)

⭐ JAVA RULE

- `abstract final class` ✕ — abstract *needs* a child, final *forbids* one
- every method of a `final` class is effectively final

Can we override static methods?

No. A static method belongs to the class and is resolved at compile time from the **reference type** — there's no object to "dispatch" on.

the Box family

```
class Box {
    static void greeting() {
        System.out.println("Hey, I am in Box class");
    }
}

class BoxWeight extends Box {
    static void greeting() {
        System.out.println("Hey, I am in BoxWeight class");
    }
}

Box b = new BoxWeight();
b.greeting();           // "Hey, I am in Box class" ← reference type wins
BoxWeight.greeting();   // "Hey, I am in BoxWeight class"
```

- static methods **are inherited** — `BoxWeight.greeting()` works even without redefining it
- redefining one in the child = **method hiding**
- instance method trying to "override" a static one (or vice-versa) ✕ compile error

☕ JAVA REALITY

Polymorphism doesn't apply to fields *either: a same-named field in the child hides the parent's (see Example 3). Rule of thumb: only* ***non-static, non-private, non-final instance methods*** *are polymorphic.*

✦ ENCAPSULATION ✦

🧠 IDEA — it's NOT just "make variables private"

```
DATA
 +
METHODS                →   ENCAPSULATION
 +                          (a capsule: inside is protected,
CONTROLLED ACCESS            outside sees only safe buttons)
```

🌍 IN REAL LIFE

An **ATM**: you can withdraw, deposit, check balance — through buttons that verify your PIN and your balance. You can **never** open the cash box and grab notes yourself.

❓ WHY? — the problem

without encapsulation

```
class Student { int marks; }

Student s = new Student();
s.marks = -50;              // nothing stops nonsense data!
```

💻 EXAMPLE — with encapsulation

Student.java

```
class Student {

    private int marks;
      ↑ private → protect the internal state

    public void setMarks(int marks) {
                 ↑ setter → the ONLY way in, and it validates
        if (marks >= 0 && marks <= 100) {
            this.marks = marks;
        }
    }

    public int getMarks() {
                ↑ getter → controlled read
        return marks;
    }
}
```

Main.java — trying to cheat

```java
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.setMarks(85);
        s.setMarks(-50);                    // rejected by the setter
        // s.marks = -50;      ✗ marks has private access
        System.out.println(s.getMarks());
    }
}

class Student {
    private int marks;
    public void setMarks(int marks) {
        if (marks >= 0 && marks <= 100) this.marks = marks;
    }
    public int getMarks() { return marks; }
}
```

▶ Output

```
85
```

✓ What you gain

- **validation** in one place
- **read-only** field → getter only (e.g. `accountNo`); **write-only** → setter only (e.g. a password)
- change the inside (store marks as `double`?) without breaking code outside
- debugging: data changes through **one door** — put a breakpoint there

📖 DEFINITION

Data hiding — keeping fields `private` so they can't be accessed directly. It is the main tool of encapsulation; encapsulation is the bigger idea: bundling data with the methods that guard it.

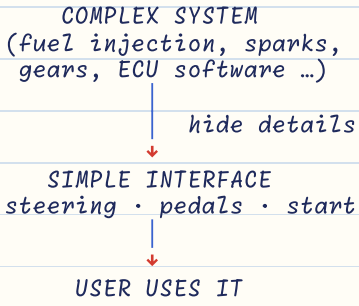
⚠️ COMMON MISTAKE

`private` fields + a setter that accepts **anything** = zero protection. Encapsulation lives in the checks, not in the keyword.

✦ ABSTRACTION ✦

🧠 IDEA

Show **what** an object does, hide **how** it does it.



🌍 IN REAL LIFE

You **drive** a car with a steering wheel and two pedals — you don't need to understand combustion. You **send** a WhatsApp message without knowing about TCP/IP.

💻 Abstraction at every level

Car.java

```java
public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();                    // the user only knows "start"
    }
}

class Car {
    public void start() {               // simple, public
        injectFuel();
        igniteSpark();
        System.out.println("Car started");
    }
    private void injectFuel()  { /* complex detail, hidden */ }
    private void igniteSpark() { /* complex detail, hidden */ }
}
```

▶ Output

```
Car started
```

- a **method** is abstraction: `Arrays.sort(arr)` — you don't care which algorithm runs inside
- a **class** with a small public API hides private helpers (`resize()` in our ArrayList, Ch 06)
- `List<Integer> list = new ArrayList<>();` — you code against **what** a List does
- **abstract classes & interfaces** (Ch 05) are Java's tools for designing with abstraction

⚠️ COMMON MISTAKE

"Abstraction = the `abstract` keyword" ×. The keyword is one tool. Abstraction is the ***idea*** of hiding complexity behind a simple interface — a plain method already does it.

✦ ENCAPSULATION vs ABSTRACTION ✦

	ENCAPSULATION	ABSTRACTION
Question it answers	**how** do I protect the data?	**what** should the user see?
View	inside (implementation level)	outside (design level)
Hides	data (internal state)	complexity (implementation details)
Achieved with	`private` fields + validating getters/setters	abstract classes, interfaces, public API methods
ATM example	locked cash box, PIN-checked operations	a screen with 4 simple buttons
Java example	`private int marks` + `setMarks()`	`List` interface, `car.start()`

ABSTRACTION = what you see

```
[Withdraw]  [Deposit]
[Balance]   [PIN]
```

ENCAPSULATION = what's locked

```
private balance
private pin
withdraw() checks pin, bal
```

💡 CORE IDEA

They work **together**: encapsulation is how you build the capsule; abstraction is the simple face the capsule shows. Good encapsulation makes abstraction possible.

⭐ CHAPTER 03 – REMEMBER

⭐ REMEMBER

- `extends` = IS-A; one parent class only; private & constructors are not inherited
- constructors run **parent first**; missing `super(...)` → hidden `super()`
- **reference type → what you can access · object type → which overridden method runs**
- overloading = compile time (different params) · overriding = runtime (same signature)
- static methods & fields are **hidden**, not overridden · `final` method = can't override
- encapsulation = data + methods + controlled access · abstraction = show what, hide how

🧪 PRACTICE – CHAPTER 03

🧪 QUICK CHECK

1) `Box b = new BoxWeight(2, 3, 4, 8); System.out.println(b.weight);` — compiles?

2) Parent has `void f(int x)`, child declares `void f(long x)`. Overloading or overriding? Which runs for `child.f(5)`?

3) `A ← B ← C` (C extends B extends A), each constructor prints its letter. Output of `new C()`?

💻 CODE PRACTICE

1) `Employee` (name, baseSalary, `double salary()`), with `Manager` (+bonus) and `Intern` (half salary) overriding `salary()`. Store all in `Employee[]` and print the payroll with **one** loop.

2) Encapsulated `BankAccount`: private balance; `deposit` / `withdraw` reject negatives and overdrafts; no `setBalance`. Show that `acc.balance = 1000000` fails to compile.

🎯 INTERVIEW THINKING

Java

```java
class A { void show() { System.out.println("A"); } }
class B extends A { void show() { System.out.println("B"); } }

A obj = new B();
((A) obj).show();
```

"A" or "B"? Why?

☑ Answers

1) × — reference type `Box` has no `weight`. 2. Overloading (different parameter type) — `f(5)` picks the exact match `f(int)`, the parent's. 3. `A B C` — parent constructors finish first. **Interview:** `B`. A cast changes only the reference type (what may be called). Which overridden method runs is always decided by the **object** — a `B`.

👨‍🏫 HOW I WOULD TEACH THIS

1) Copy-paste Box → BoxWeight → BoxPrice: feel the pain
2) Parent/child analogy → `extends`
3) Print the constructor chain, ask for the order
4) `Shapes[]` loop → polymorphism in one line
5) Board: "reference decides ACCESS, object decides BEHAVIOUR"
6) `marks = -50` bug → encapsulation; ATM → abstraction

🔗 NEXT →

`protected`, default, `private`, `public` — who exactly can see what across packages? And who is the parent of every class, even `Box`? → **Access control & the `Object` class** (Chapter 04).

04 ACCESS CONTROL · PACKAGES · OBJECT CLASS

private · default · protected · public · built-in packages · Object · toString · equals · hashCode · getClass · instanceof

✦ ACCESS MODIFIERS ✦

🧠 IDEA

An access modifier decides **who is allowed to see a member** (field, method, constructor, nested class).

Java has 4 levels — from most locked to most open:

`private` → *default (no keyword)* → `protected` → `public`

🌍 IN REAL LIFE

*default access has **no keyword** — you just write nothing*

your house

private	→ your diary	only YOU
default	→ the family fridge	everyone in the SAME house (package)
protected	→ family heirlooms	the house + your CHILDREN, even in another city (other package)
public	→ the doorbell	ANYONE on the street

📋 THE ACCESS MATRIX

🔴 MUST KNOW

	SAME CLASS	SAME PACKAGE	CHILD in other package	OTHER (world)
private	✓	×	×	×
default	✓	✓	×	×
protected	✓	✓	✓ *	×
public	✓	✓	✓	✓

* only through inheritance — see the next page!

	class	package	subclass (same pkg)	subclass (other pkg)	world
`public`	✓	✓	✓	✓	✓
`protected`	✓	✓	✓	✓ *	×
default	✓	✓	✓	×	×
`private`	✓	×	×	×	×

Class `A` — the access demo

com/kunal/access/A.java

```java
package com.kunal.access;

public class A {
    protected int num;      // package + children
    String name;            // default → package only
    int[] arr;

    public int getNum() { return num; }            // public door
    public void setNum(int num) { this.num = num; }

    public A(int num, String name) {
        this.num = num;
        this.name = name;
        this.arr = new int[num];
    }
}
```

From `com.kunal.singleton.Main` *(other package,* ***not*** *a child):* `a.getNum()` ✓ · `a.num` × · `a.name` ×.

⭐ JAVA RULE

- a ***top-level*** class can only be `public` or default — `private class A {}` at top level × (nested classes may use all four)
- a `public` class must live in a file with the same name
- overriding can't ***reduce*** access (Ch 03): `public` → `protected` ×

When to use which?

- fields → `private` (+ getters/setters when needed) — encapsulation
- methods other code needs → `public`
- helpers used only inside the class → `private`
- "hooks" meant for subclasses → `protected`
- package-internal helpers → default
- constructor → `private` when objects must be controlled (Singleton)

✦ THE protected NUANCE ✦

HIGHLY IMPORTANT

IDEA

In **another package**, a subclass can use a protected member **only through inheritance** — on itself (or objects of its own type / its subtypes). It can **not** poke at the protected members of an arbitrary parent object.

one/Base.java

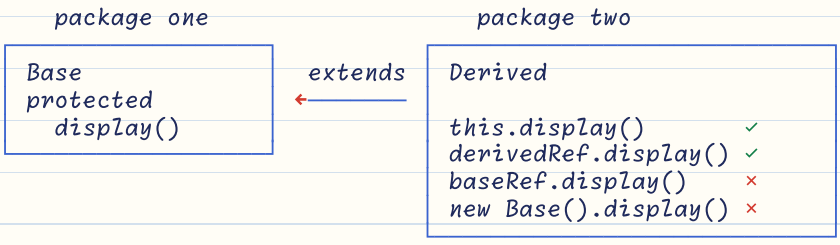
```
package one;

public class Base {
    protected void display() { System.out.println("in Base"); }
}
```

two/Derived.java

```
package two;
import one.Base;

public class Derived extends Base {
    void test(Base b, Derived d) {
        display();                  //✓ inherited — called on this
        d.display();                //✓ a Derived reference
        new Derived().display();    //✓ a Derived object
        b.display();                //✗ has protected access in Base
        new Base().display();       //✗ has protected access in Base
    }
}
```



WHY this rule?

If `b` is just "some `Base`", it might really be `OtherChild` — a sibling class that `Derived` knows nothing about. Letting `Derived` tamper with a stranger's protected internals would be unsafe. Its **own** kind (`Derived` and its subclasses)? Fine — that's shared family knowledge.

🎯 INTERVIEW TRAP #17

"protected = visible to all subclasses, everywhere" — only **through inheritance**. In a different package, `new Base().display()` does **not** compile, even inside a subclass of `Base`. (In the **same** package, protected behaves like default → everything ✓.)

🎯 INTERVIEW TRAP #18

A **default** (package-private) member is **not inherited** by a subclass in another package at all: calling it there → "cannot find symbol".

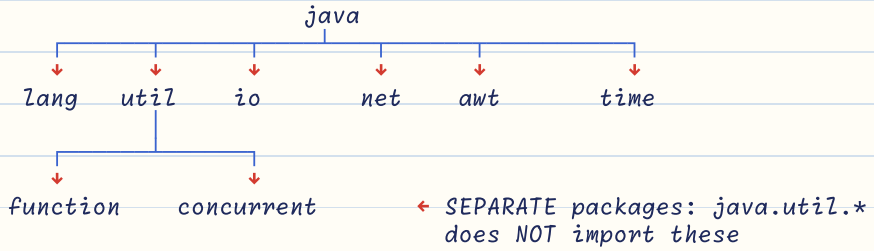
✦ BUILT-IN PACKAGES ✦

IDEA

The JDK ships thousands of ready classes, organised in packages. Know the big ones:

package	what lives there	examples
`java.lang`	core language — **auto-imported**	`String`, `Math`, `System`, `Object`, `Integer`, `Thread`, exceptions
`java.util`	collections & utilities	`ArrayList`, `HashMap`, `Scanner`, `Arrays`, `Collections`, `Random`
`java.io`	input / output, files	`File`, `BufferedReader`, `InputStreamReader`, `PrintStream`
`java.net`	networking	`URL`, `Socket`, `URI` (+ `java.net.http.HttpClient`)
`java.awt`	old GUI toolkit	`Frame`, `Button`, `Color`, `Graphics`
`javax.swing`	GUI built on AWT	`JFrame`, `JButton`, `JLabel`
`java.applet`	browser applets	`Applet` — history now (see below)

java is a tree of packages



JAVA REALITY

Applets are gone. The Applet API was deprecated in JDK 9, deprecated for removal in JDK 17 and **removed in JDK 26** (JEP 504). Mention it only as history; no browser runs applets today.

CONNECT THIS WITH

`System.out` (Ch 02) = class `System` from `java.lang` — that's why you never import it.

`Scanner` is in `java.util` — that's why you always do.

✦ THE `Object` CLASS ✦

🧠 IDEA

`java.lang.Object` is the **root of every class hierarchy**. A class that extends nothing silently extends `Object`.

```
Object          ← java.lang.Object — the root
  ↑
Person          class Person { }
  ↑                 = class Person extends Object { }
Student
  ↑
CollegeStudent  still an Object, 3 levels down
```

❓ WHY does it matter?

- **every** object (arrays too!) gets the same basic methods for free
- `Object o = anything;` works → `println(Object x)` can print *any* object
- before generics, collections stored `Object` — Chapter 06 starts exactly from that problem

📋 The methods every object has

method	job	override it?
`toString()`	text form; default `ClassName@hexHash`	✓ almost always
`equals(Object o)`	logical equality; default = `==`	✓ for value-like classes
`hashCode()`	int "bucket number" for `HashSet` / `HashMap`; default identity-based	✓ **whenever** `equals` is
`getClass()`	the runtime class of the object	× it's `final`
`clone()`	field-by-field copy (`protected`, needs `Cloneable`)	Ch 06
`finalize()`	old cleanup hook	× deprecated
`wait()`, `notify()`, `notifyAll()`	thread coordination	× `final`

ObjectDemo.java

```java
public class ObjectDemo {
    int num;
    float gpa;

    ObjectDemo(int num, float gpa) { this.num = num; this.gpa = gpa; }

    @Override
    public boolean equals(Object obj) {
        return this.num == ((ObjectDemo) obj).num;    // "same num = equal"
    }

    public static void main(String[] args) {
        ObjectDemo obj1 = new ObjectDemo(12, 56.8f);
        ObjectDemo obj2 = new ObjectDemo(12, 78.9f);
        System.out.println(obj1 == obj2);          // two different objects
        System.out.println(obj1.equals(obj2));     // our rule says equal
        System.out.println(obj1.getClass().getName());
    }
}
```

▶ *Output*

```
false
true
ObjectDemo
```

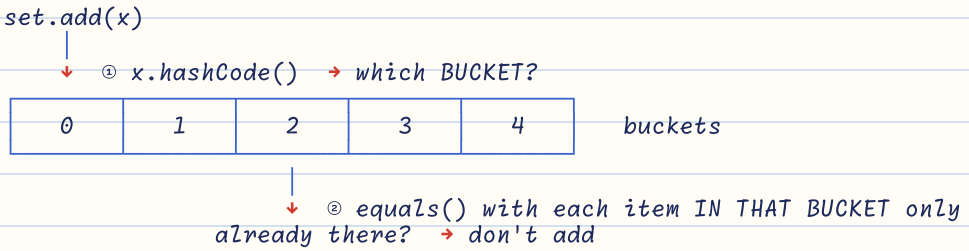
(Inside package `com.kunal.access` the last line prints `com.kunal.access.ObjectDemo` — the fully qualified name.)

⚠ **COMMON MISTAKE**

*This quick `equals` would crash on `obj1.equals("hi")` (ClassCastException) and on `null`. And it overrides `equals` **without** `hashCode` — which breaks `HashSet` / `HashMap` (next page).*

✦ equals() + hashCode() ✦

🧠 IDEA — how a HashSet thinks



*If two "equal" objects land in **different buckets**, `equals()` is never even asked → duplicates ×.*

💻 EXAMPLE – doing it right

Main.java

```java
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        Set<Student> set = new HashSet<>();
        set.add(new Student(15, "Kunal"));
        set.add(new Student(15, "Kunal"));    // the "same" student again
        System.out.println(set.size());
        System.out.println(new Student(15, "Kunal"));
    }
}

class Student {
    private final int rno;
    private final String name;

    Student(int rno, String name) { this.rno = rno; this.name = name; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                    // same object
        if (!(o instanceof Student)) return false;     // null / other type
        Student other = (Student) o;
        return rno == other.rno && Objects.equals(name, other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(rno, name);          // SAME fields as equals
    }

    @Override
    public String toString() {
        return "Student{rno=" + rno + ", name=" + name + "}";
    }
}
```

▶ Output

```
1
Student{rno=15, name=Kunal}
```

Delete `hashCode()` and run again → `2` — two "equal" students both stored.

⭐ JAVA RULE

The contract 1. `a.equals(b)` is true → `a.hashCode() == b.hashCode()` **must** be true 2. same hash code does **not** mean equal (collisions are allowed) 3. `equals` must be reflexive, symmetric, transitive, consistent — and `x.equals(null)` is `false` 4. override **both or neither**, using the **same fields**

🎯 INTERVIEW TRAP #19

Override `equals()` without `hashCode()` → it compiles, it runs, and it silently breaks every `HashSet` / `HashMap` : `contains()` says false for an "equal" object, duplicates appear.

== vs .equals()

	==	.equals()
What is it	operator	method (from `Object`)
On primitives	compares **values** ✓	× can't call methods on primitives
On references	**same object?** (identity)	**same content?** — if the class overrides it
Default in `Object`	identity	identity too (until overridden!)
`String`	same object in memory?	same characters?
Customisable	×	✓ override it
With `null`	`x == null` is safe	`null.equals(x)` → NullPointerException

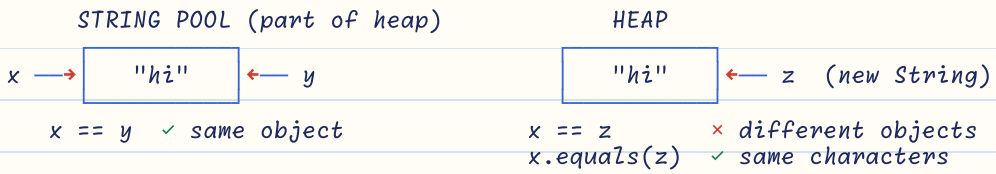
verified on Java 25

```java
public class Main {
    public static void main(String[] args) {
        String x = "hi";
        String y = "hi";                // same pooled literal
        String z = new String("hi");    // forced new object
        System.out.println((x == y) + " " + (x == z) + " " + x.equals(z));

        Integer a = 127, b = 127, c = 128, d = 128;
        System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));
    }
}
```

▶ Output

```
true false true
true false true
```



🎯 INTERVIEW TRAP #20

`Integer` values from −128 to 127 are cached, so `==` seems to work in small tests and fails at 128.

Rule: **objects** → `.equals()`, primitives → `==`.

⚠️ COMMON MISTAKE

`if (name.equals("Kunal"))` crashes when `name` is `null`. Safer: `"Kunal".equals(name)` or `Objects.equals(name, "Kunal")`.

✦ getClass() & instanceof ✦

IDEA

→ `getClass()` → the **exact** runtime class of the object

→ `x instanceof T` → **IS-A** test: is the object a `T` or any subtype of `T`?

Main.java

```
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        System.out.println(a.getClass().getSimpleName()); // real type
        System.out.println(a instanceof Dog);
        System.out.println(a instanceof Animal);
        System.out.println(a instanceof Object);

        Animal b = null;
        System.out.println(b instanceof Animal);              // null → false

        if (a instanceof Dog d) {       // Java 16+: test + cast in one step
            d.bark();
        }
    }
}

class Animal { }
class Dog extends Animal { void bark() { System.out.println("Woof"); } }
```

▶ Output

```
Dog
true
true
true
false
Woof
```

	`getClass()`	`instanceof`
Answers	"exactly which class?"	"is it a kind of ...?"
`Dog` **object vs** `Animal`	`Dog` ≠ `Animal`	✓ true
On `null`	NullPointerException	`false`

unrelated types

```
public class Test {
    public static void main(String[] args) {
        String s = "hello";
        System.out.println(s instanceof Integer);
    }
}
```

💥 *compile error*

```
error: incompatible types: String cannot be converted to Integer
```

🎯 INTERVIEW TRAP #21

`null instanceof Anything` is always **false** — no exception. That's why `instanceof` is the classic first line of `equals()`.

⭐ CHAPTER 04 — REMEMBER

⭐ REMEMBER

- `private` < *default* < `protected` < `public`; top-level classes: only `public` or default
- *protected in another package = only* **through inheritance** *(on your own type)*
- `java.lang` *is auto-imported; sub-packages are separate packages*
- *every class extends* `Object` → `toString`, `equals`, `hashCode`, `getClass` *for free*
- *override* `equals` → *override* `hashCode` *(same fields)*
- `==` *= same object ·* `.equals()` *= same content ·* `instanceof` *= IS-A*

🧪 PRACTICE — CHAPTER 04

🧪 QUICK CHECK

1) `two.Derived extends one.Base`. *Inside* `Derived`, *does* `new Base().display()` *compile if* `display()` *is protected?*
2) *Two* `Student`*s are equal by* `equals()`, *but* `hashCode()` *isn't overridden. Add both to a* `HashSet` *— size?*
3) `String a = "java"; String b = new String("java");` → `a == b`? `a.equals(b)`?

💻 CODE PRACTICE

1) `Point(x, y)`: *override* `toString`, `equals`, `hashCode`. *Prove it works as a* `HashMap` *key:* `map.get(new Point(1, 2))`.
2) *Packages* `bank` *and* `app`: `Account` *with a private balance, a protected* `applyInterest()`, *a default* `audit()`. *In* `app`, `SavingsAccount extends Account` *— list which calls compile and why.*

🎯 INTERVIEW THINKING

A class overrides `equals()` *to compare* `id`, *but not* `hashCode()`. *Everything "works" in your tests with* `ArrayList`. *When does it break, and how would the bug look in production?*

☑ *Answers*

1) × — outside the package, protected works only through inheritance (`this` / `Derived` references). 2. Almost always **2** — different default hash codes → different buckets. 3. `false`, `true`. **Interview:** `ArrayList.contains` uses only `equals`, so it looks fine. The moment the objects go into a `HashSet` or become `HashMap` keys, lookups miss and duplicates appear — "the cache never hits", "the set has two copies of the same user".

👨‍🏫 HOW I WOULD TEACH THIS

1) House analogy: diary · fridge · heirlooms · doorbell
2) Fill the access matrix together on the board
3) Two-package demo — let `new Base().display()` fail live
4) Print an object without `toString()` — "what's that @1b6d...?"
5) HashSet duplicates → motivate `hashCode()`
6) Quiz: `getClass()` vs `instanceof` on a `Dog`

🔗 NEXT →

`Object` gives every class common methods with default behaviour. What if a parent wants to **force** every child to write its own `career()`? And how do we get multiple inheritance without the diamond problem? → **Abstract classes & interfaces** (Chapter 05).

05 ABSTRACT CLASS · INTERFACE · ANNOTATIONS

multiple inheritance problem · abstract classes · interfaces · default & static methods · extending & nested interfaces · @Override

✦ THE MULTIPLE INHERITANCE PROBLEM ✦

? WHY this chapter?

Two needs pull in opposite directions:

1) A parent wants to **force** children to write certain methods ("every child must define `career()`"), without writing them itself.
2) A class wants behaviour from **many** parents (a `Car` is an engine-thing, a brake-thing, a media-thing) — but Java allows only **one** parent class because of the diamond problem.

```
        A greet() { "A" }        ← code + state in both parents
        ↙              ↘
B greet() {"B"}    C greet() {"C"}
        ↘              ↙
              D
                    d.greet() → "B" or "C" ??    × classes

FIX: parents that only PROMISE a method (no code, no state)
     → nothing can conflict → many parents become safe ✓
```

💡 CORE IDEA

Abstract class = a *partly built* parent: some code, some promises. ***Interface*** = a *pure contract*: "whoever signs me must provide these methods". A class may extend one class but sign ***many*** contracts.

✦ ABSTRACT CLASSES ✦

🧠 IDEA

An `abstract` class is an **incomplete** class. It may declare `abstract` methods — methods with **no body** — that every concrete child **must** implement.

🌍 IN REAL LIFE

A parent says: "Every child of mine must choose a career and a partner — but I won't decide for them." The rule is fixed; the details are each child's responsibility.

📖 DEFINITION

- *abstract method* — declared without a body: `abstract void career();` — "subclass responsibility"
- *abstract class* — declared with `abstract`; can't be instantiated; may mix abstract and concrete methods

EXAMPLE

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Son son = new Son(30);
        son.career();
        son.normal();

        Parent daughter = new Daughter(28);   // abstract type as ref ✓
        daughter.career();

        Parent.hello();                       // static method ✓
        // Parent mom = new Parent(45);     × Parent is abstract
    }
}

abstract class Parent {
    int age;
    final int VALUE;

    Parent(int age) {                 // constructor ✓ (runs via super)
        this.age = age;
        VALUE = 32456789;
    }

    static void hello() { System.out.println("hey"); }
    void normal() { System.out.println("this is a normal method"); }

    abstract void career();                   // no body → child decides
    abstract void partner();
}

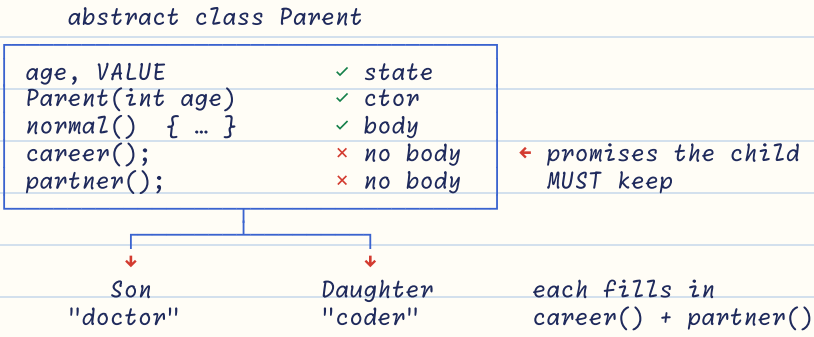
class Son extends Parent {
    Son(int age) { super(age); }
    @Override
    void career()  { System.out.println("I am going to be a doctor"); }
    @Override
    void partner() { System.out.println("I love Pepper Potts"); }
}

class Daughter extends Parent {
    Daughter(int age) { super(age); }
    @Override
    void career()  { System.out.println("I am going to be a coder"); }
    @Override
    void partner() { System.out.println("I love Iron Man"); }
}
```

▶ Output

```
I am going to be a doctor
this is a normal method
I am going to be a coder
hey
```

🧠 VISUALIZE IT



❓ Why can't we write `new Parent(45)`?

*It's incomplete — `mom.career()` would have **no code to run**. The compiler refuses: "Parent is abstract; cannot be instantiated".*

❓ *"No objects… so why does an abstract class have a constructor?"*

`new Son(30)` *builds **one** object that contains a* `Parent` *part.* `super(age)` *runs* `Parent`*'s constructor to initialise that part (*`age`*,* `VALUE`*). A constructor **initialises**; only* `new` ***creates***.

✦ ABSTRACT CLASS – THE RULES ✦

inside an abstract class …	*allowed?*
abstract methods (zero or more)	✓
concrete methods, `final` ***methods***	✓
constructors	✓ — *run through* `super(...)`
instance, static and `final` ***fields***	✓
static methods → `Parent.hello()`	✓
`abstract static void f();`	× *illegal combination of modifiers*
`abstract Parent();` ***(abstract constructor)***	× *modifier abstract not allowed here*
`new Parent(…)`	× *cannot be instantiated*
`abstract final class`	× *illegal combination of modifiers*

❓ Why no `abstract static`?

A static method is called on the ***class****:* `Parent.career()`*. If it were abstract there'd be an* ***empty*** *method to call — and static methods can't be overridden, so no child could ever fill it in.*

⚠️ Forgot one abstract method?

Son.java

```
abstract class Parent {
    abstract void career();
    abstract void partner();
}

class Son extends Parent {
    void career() { }        // partner() forgotten
}
```

> 💥 *compile error*
>
> `error: Son is not abstract and does not override abstract method partner() in Parent`

Either implement ***all*** *abstract methods, or declare the child* `abstract` *too.*

🎯 INTERVIEW TRAP #22

"An abstract class must contain an abstract method" × — zero abstract methods is legal (you still can't instantiate it). The reverse **is** true: a class with even one abstract method **must** be declared abstract.

🎯 INTERVIEW TRAP #23

"Multiple inheritance via abstract classes?" × — an abstract class is still a **class**. `class C extends A, B` is illegal whether A and B are abstract or not.

✦ INTERFACES ✦

🧠 IDEA

An interface is a **contract**: it lists what a class must be able to do, never how. A class **signs** it with `implements`.

🌍 IN REAL LIFE

✧ abstract class = IS-A
interface = CAN-DO

A **power socket**: any appliance with the right plug works — the socket doesn't care if it's a phone charger or a kettle. Or a job description: "every Driver must `start()`, `stop()`, `brake()`" — each driver does it their own way.

📖 DEFINITION

An `interface` is a reference type containing abstract method signatures (plus, since Java 8, `default` and `static` methods) and constants. A class can implement **any number** of interfaces.

✍️ SYNTAX — what Java adds silently

Java

```java
interface Engine {
    int PRICE = 78000;      // = public static final int PRICE = 78000;
    void start();           // = public abstract void start();
}
```

💻 EXAMPLE — one car, three contracts

Main.java

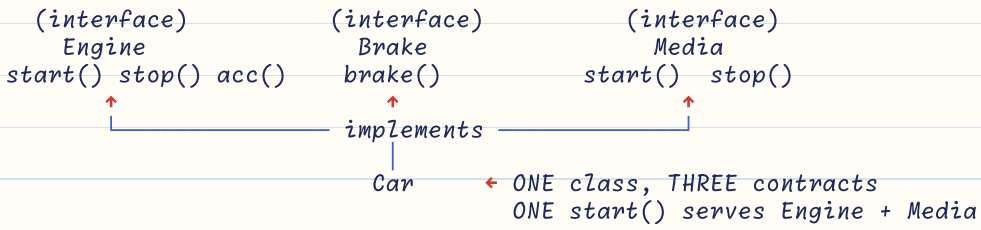
```java
public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();
        car.brake();
        System.out.println(Engine.PRICE);
    }
}

interface Engine {
    int PRICE = 78000;
    void start(); void stop(); void acc();
}
interface Brake { void brake(); }
interface Media { void start(); void stop(); }    // same start()/stop()!

class Car implements Engine, Brake, Media {
    @Override
    public void start() { System.out.println("I start like a normal Car"); }
    @Override
    public void stop()  { System.out.println("I stop like a normal Car"); }
    @Override
    public void acc()   { System.out.println("I accelerate"); }
    @Override
    public void brake() { System.out.println("I brake like a normal Car"); }
}
```

▶ Output

```
I start like a normal Car
I brake like a normal Car
78000
```



⭐ JAVA RULE

- implementing methods must be `public` — `void start() { }` → "attempting to assign weaker access privileges; was public"
- interface variables are constants → `PRICE = 1;` in a class → "cannot assign a value to static final variable PRICE"
- no constructors, no instance fields → an interface holds **no state**
- implement **every** abstract method, or the class must be `abstract`

🎯 INTERVIEW TRAP #24

Same abstract method in two interfaces (`Engine.start()`, `Media.start()`) is **not** a diamond problem — neither has code, so **one** implementation satisfies both.

✦ INTERFACE REFERENCES ✦

Same rule as Ch 03

Java

```
Engine car = new Car();        // interface type as reference ✓
car.acc();                     //✓ Engine declares acc()
car.brake();                   //× Engine has no brake()
Media carMedia = new Car();
carMedia.stop();               // runs Car's stop() — the OBJECT decides
```

Reference type → what you may call · object type → which code runs.

"Separate classes, same interface" — plug-in parts

Main.java

```
public class Main {
    public static void main(String[] args) {
        NiceCar car = new NiceCar();
        car.start();
        car.startMusic();
        car.upgradeEngine();       // swap the part at RUNTIME
        car.start();
    }
}

interface Engine { void start(); }
interface Media  { void start(); }

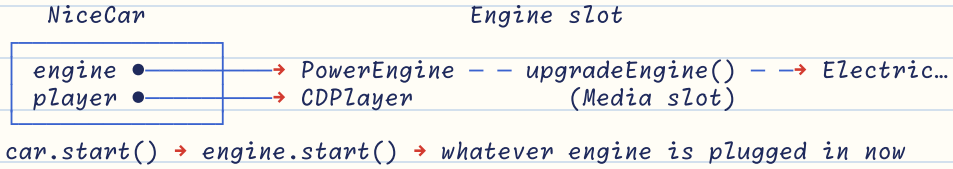
class PowerEngine implements Engine {
    public void start() { System.out.println("Power engine start"); }
}
class ElectricEngine implements Engine {
    public void start() { System.out.println("Electric engine start"); }
}
class CDPlayer implements Media {
    public void start() { System.out.println("Music start"); }
}

class NiceCar {
    private Engine engine = new PowerEngine(); // HAS-A Engine (any!)
    private Media player = new CDPlayer();     // HAS-A Media

    void start()         { engine.start(); }
    void startMusic()    { player.start(); }
    void upgradeEngine() { engine = new ElectricEngine(); }
}
```

▶ *Output*

```
Power engine start
Music start
Electric engine start
```



💡 CORE IDEA

`Car implements Engine` says "a car **IS** an engine" — odd. `NiceCar` says "a car **HAS** an engine" — natural, and the engine is swappable. This is **composition**, and coding against the **interface** (`Engine`) instead of a class is what makes the swap possible.

✦ EXTENDING INTERFACES ✦

Main.java — extendDemo

```java
public class Main implements B {
    public void fun()   { System.out.println("fun from A's contract"); }
    public void greet() { System.out.println("greet from B"); }

    public static void main(String[] args) {
        Main obj = new Main();
        obj.fun();
        obj.greet();
        A ref = obj;                // B extends A → Main IS-A A too
        ref.fun();
    }
}

interface A { void fun(); }
interface B extends A { void greet(); }     // B = A's methods + greet()
```

▶ Output

```
fun from A's contract
greet from B
fun from A's contract
```

An interface `extends` interfaces (even **several**: `interface C extends A, B`). The implementing class owes **all** inherited methods.

✦ `default` & `static` METHODS ✦ (Java 8+)

❓ WHY default methods?

Add a method to an interface used by 1000 classes → all 1000 break. A `default` method ships **with a body**, so old classes keep compiling. (That's how `forEach()` was added to every collection.)

Main.java — extendDemo2

```java
public class Main implements A, B {
    @Override
    public void fun() {           // MUST override: A and B both give fun()
        A.super.fun();            // optionally reuse one parent's version
        System.out.println("Main's own fun");
    }

    public static void main(String[] args) {
        new Main().fun();
        A.greeting();             // static → call via the INTERFACE name
        // Main.greeting();    × static interface methods: not inherited
    }
}

interface A {
    default void fun() { System.out.println("I am in A"); }
    static void greeting() { System.out.println("Hey I am static method"); }
}

interface B {
    default void fun() { System.out.println("I am in B"); }
}
```

▶ Output

```
I am in A
Main's own fun
Hey I am static method
```

⭐ JAVA RULE

Default-method conflicts 1. **class wins** — a method in the class (or its superclass) beats any interface default 2. **more specific interface wins** — `B extends A`, both define `fun()` → B's 3. two **unrelated** interfaces with the same default → compile error ("types A and B are incompatible") until you override it

🎯 INTERVIEW TRAP #25

Static interface methods are **not inherited** — not by implementing classes, not by sub-interfaces. `A.greeting()` ✓ · `obj.greeting()` / `Main.greeting()` ×. They must have a body.

☕ JAVA REALITY

🟡 GOOD TO KNOW Since Java 9 interfaces may also have `private` methods — helpers shared by default methods. Everything else in an interface is implicitly `public`.

✦ NESTED INTERFACES ✦

🧠 IDEA

An interface declared **inside** a class (or another interface) — grouped with the class it belongs to. Use it as `Outer.InterfaceName`.

Main.java — nested

```
public class Main {
    public static void main(String[] args) {
        A.NestedInterface obj = new B();   // Outer.Inner name
        System.out.println(obj.isOdd(6));
        System.out.println(obj.isOdd(7));
    }
}

class A {
    public interface NestedInterface {     // a member of class A
        boolean isOdd(int num);
    }
}

class B implements A.NestedInterface {
    @Override
    public boolean isOdd(int num) {
        return (num & 1) == 1;             // last bit 1 → odd
    }
}
```

▶ *Output*

```
false
true
```

	top-level interface	nested interface
allowed access	`public` or default	`public`, `protected`, `private` or default
static?	—	implicitly `static` (no outer object needed)
named as	`NestedInterface`	`A.NestedInterface`

🔗 CONNECT THIS WITH

`Map.Entry<K, V>` (Ch 07) is a real nested interface —

`for (Map.Entry<String, Integer> e : map.entrySet())`.

✦ ABSTRACT CLASS vs INTERFACE ✦

MUST KNOW

	ABSTRACT CLASS	INTERFACE
Keyword	`abstract class`	`interface`
A class uses it with	`extends` — **one** only	`implements` — **many**
Methods	abstract + concrete, any access	abstract (implicitly `public`) + `default` + `static` + `private`
Variables	anything: instance, static, final, non-final	only `public static final` constants
Constructors	✓ (run via `super`)	×
Holds state (instance fields)	✓	×
Multiple inheritance	×	✓ — interfaces can even extend many interfaces
Can implement / extend the other	✓ abstract class can implement interfaces	× interface can't extend a class
Relationship	**IS-A** — a family of related classes	**CAN-DO** — a capability / contract
Use when	related classes share code **and** state	unrelated classes share a **capability**
JDK examples	`Number` (← `Integer`, `Double`), `AbstractList`	`Comparable`, `Runnable`, `List`

which one?

```
"they ARE the same kind of thing, sharing code/state" → ABSTRACT CLASS
      Animal ← Dog, Cat                    (common: name, age, eat())

"they CAN DO the same thing, otherwise unrelated"     → INTERFACE
      Comparable ← String, Integer, Student
      Flyable    ← Bird, Plane, Superman
```

🎯 INTERVIEW TRAP #26

"Interfaces have default methods now, so they're the same as abstract classes" ×. Interfaces still have **no state** (no instance fields), **no constructors**, and all non-private members are **public** — and a class can implement many of them.

⭐ REMEMBER

- can't instantiate either one — both work as **reference types**
- abstract class = partial implementation for a family; interface = contract for anyone
- design tip: start with an interface; add an abstract class when implementations share code

✦ ANNOTATIONS ✦

🧠 IDEA

An annotation (`@Something`) is **metadata** stuck onto code. It doesn't change what the code does by itself — the **compiler, tools and frameworks** read it and act on it.

🌍 IN REAL LIFE

Labels on a parcel: "FRAGILE", "THIS SIDE UP". The contents don't change — but the delivery person handles the box differently.

annotation	tells ...	effect
`@Override`	compiler: "I mean to override"	× error if nothing is overridden
`@Deprecated`	users: "don't use this any more"	warning wherever it's used
`@FunctionalInterface`	compiler: "exactly ONE abstract method"	× error if 0 or 2+ (Ch 06 lambdas)
`@SuppressWarnings("unchecked")`	compiler: "I know, be quiet"	hides that warning (generic casts, Ch 06)

💻 Why `@Override` saves you

Student.java

```
class Student {
    @Override
    public String tostring() {      // typo: small "s"
        return "Student";
    }
}
```

> 💥 compile error — a GOOD error
>
> `error: method does not override or implement a method from a supertype`

Without `@Override` this compiles silently, and `println(student)` keeps printing `Student@1b6d3586` — a bug you'd hunt for an hour.

☕ JAVA REALITY

Frameworks run on annotations: `@Test` (JUnit), `@Entity`, `@Autowired`, `@GetMapping` (Spring). You can define your own with `@interface` — good to know, not needed for this playlist.

⭐ CHAPTER 05 – REMEMBER

⭐ REMEMBER

- ⭐ abstract class: can't `new`; may have constructors, state, concrete & static methods
- ⭐ abstract method: no body → children implement it or become abstract
- ⭐ × `abstract static` · × abstract constructor · × `abstract final`
- ⭐ interface = contract: methods `public abstract`, variables `public static final`
- ⭐ many interfaces per class → multiple inheritance of ***type***, no diamond of state
- ⭐ default conflicts: class wins → specific interface wins → else override (`X.super.m()`)
- ⭐ static interface methods: call via the interface, never inherited

🧪 PRACTICE – CHAPTER 05

🧪 QUICK CHECK

1) Can an abstract class have a constructor? Who calls it, and when?
2) `interface Shape { int SIDES = 0; }` — can an implementing class write `SIDES = 4;`?
3) `class C implements A, B` where both interfaces have `default void hi()` — does it compile?

💻 CODE PRACTICE

1) Abstract `Shape` with abstract `area()` and concrete `describe()`; `Circle` and `Rectangle` extend it. Store both in `Shape[]` and print the areas in one loop.
2) Interfaces `Playable` and `Recordable`; `Phone` implements both. A `MusicSystem` ***has-a*** `Playable` and can swap it at runtime.

🎯 INTERVIEW THINKING

"Since Java 8 interfaces can have method bodies. So why do abstract classes still exist?"

☑ Answers

1) ✓ — it runs through `super(...)` when a child object is created, to initialise the parent part. 2. × — interface fields are `public static final`. 3. × — unrelated duplicate defaults; `C` must override `hi()` (and may call `A.super.hi()`). ***Interview:*** interfaces can't hold ***state*** (instance fields) or have ***constructors***, and their members are public. An abstract class is a partial implementation with shared fields and protected helpers for a family of related classes.

👨‍🏫 HOW I WOULD TEACH THIS

1) Recall the diamond problem from Ch 03
2) "Parent forces career()" story → `abstract`
3) Try `new Parent()` — read the error together
4) Interface = contract; `Car implements Engine, Brake, Media`
5) Swap engines in `NiceCar` → power of interface references
6) Build the comparison table with the class; quiz on default conflicts

🔗 NEXT →

Contracts make code flexible — but our home-made list stores **only** `int`. One class that works for *any* type, still type-safe? → **Generics** (Chapter 06).

06 GENERICS · LAMBDAS · EXCEPTIONS · CLONING

custom ArrayList · generics · bounds & wildcards · Comparable/Comparator · lambdas · try/catch/finally · throw/throws · checked/unchecked · shallow & deep copy

✦ BUILD YOUR OWN ArrayList ✦

? WHY? — the problem

Arrays are **fixed size**. `int[] a = new int[3];` then a 4th item → × `ArrayIndexOutOfBoundsException`.
We want a list that just... grows.

Fixed array
↓
Capacity problem
↓
Resize
↓
Copy old items
↓
Custom ArrayList

`int[] a = new int[3];` `a[3] = 9` × out of bounds

1	2	3

full!

make a new array, 2× the size

1	2	3	4		

← then add 4

add() hides all this → the list feels "infinite"

💻 EXAMPLE (compact version)

CustomArrayList.java

```java
import java.util.Arrays;

public class CustomArrayList {
    private int[] data = new int[10];    // DEFAULT_SIZE = 10
    private int size = 0;                // item count AND next free index

    public void add(int num) {
        if (size == data.length) resize();    // full? grow first
        data[size++] = num;
    }

    private void resize() {              // private → hidden detail
        int[] temp = new int[data.length * 2];
        for (int i = 0; i < data.length; i++) {
            temp[i] = data[i];           // copy old items
        }
        data = temp;                     // old array → garbage
    }

    public int remove()       { return data[--size]; }    // last item
    public int get(int index) { return data[index]; }
    public int size()         { return size; }

    @Override
    public String toString() {
        return Arrays.toString(Arrays.copyOf(data, size))
               + " size=" + size + " capacity=" + data.length;
    }

    public static void main(String[] args) {
        CustomArrayList list = new CustomArrayList();
        for (int i = 0; i < 14; i++) list.add(2 * i);
        System.out.println(list);
    }
}
```

▶ *Output*

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26] size=14 capacity=20
```

🔗 CONNECT THIS WITH

`resize()` is **private** → the user sees only `add/get/remove/size` = **abstraction + encapsulation** (Ch 03) in a real class. The real `java.util.ArrayList` does the same, growing by ~1.5×.

⚠️ The new problem

It stores **only** `int`. Strings? Students? Write `StringArrayList`, `StudentArrayList` … × copy-paste. Use `Object[]`? Then anything goes in and you cast everything coming out ×. We need **one** class that works for any type **and** stays type-safe → **generics**.

✦ GENERICS ✦

❓ WHY? — the problem first

raw ArrayList — no type checking

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList list = new ArrayList();    // raw type: holds Object
        list.add(10);
        list.add("Hello");                   // the compiler can't stop this
        for (Object o : list) {
            Integer n = (Integer) o;         // boom for "Hello" — at RUNTIME
            System.out.println(n * 2);
        }
    }
}
```

💥 runtime

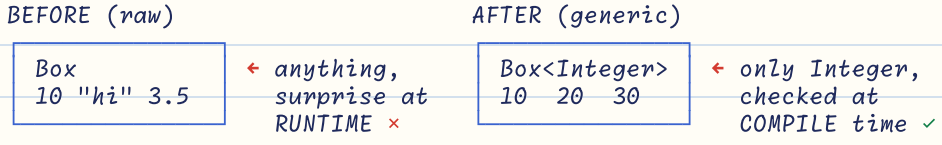
```
20
Exception in thread "main" java.lang.ClassCastException: class java.lang.String cannot be
cast to class java.lang.Integer
```

with generics

```
ArrayList<Integer> list = new ArrayList<>();  // <> "diamond": inferred
list.add(10);
list.add("Hello");      // × COMPILE error — caught before the program runs
int n = list.get(0);    // ✓ no cast needed
```

🌍 IN REAL LIFE

A **labelled storage box**: "ONLY BOOKS". Nobody can drop a sandwich in it, and whatever you take out is surely a book.



Generics → type safety → fewer runtime type errors → less casting

📖 DEFINITION

Generics let a class, interface or method take a type parameter (`<T>`). The compiler fills in the real type at each use and checks it. Common names: `T` type · `E` element · `K` key · `V` value · `N` number.

💻 EXAMPLE — our list, now generic

CustomGenArrayList.java

```java
import java.util.Arrays;

public class CustomGenArrayList<T> {          // T = type parameter
    private Object[] data = new Object[10];   // new T[10] × (see below)
    private int size = 0;

    public void add(T item) {
        if (size == data.length) {
            data = Arrays.copyOf(data, data.length * 2);
        }
        data[size++] = item;
    }

    @SuppressWarnings("unchecked")
    public T get(int index) { return (T) data[index]; }   // only Ts go in

    public int size() { return size; }

    public static void main(String[] args) {
        CustomGenArrayList<String> names = new CustomGenArrayList<>();
        names.add("Kunal");
        names.add("Rahul");
        // names.add(42);          × compile error
        String first = names.get(0);      // no cast at the call site
        System.out.println(first + " " + names.size());
    }
}
```

▶ Output

```
Kunal 2
```

☕ JAVA REALITY

Type erasure: **generics exist only for the compiler**. At runtime `T` is erased to `Object` (or its bound), so `new T[10]` ("generic array creation"), `new T()` and `instanceof T` are impossible. Hence `Object[]` + a cast inside the class.

🎯 INTERVIEW TRAP #27

Generics work with **objects only**: `List<int>` × ("unexpected type") → `List<Integer>` + autoboxing (Ch 01 wrappers).

✍️ Generic methods & interfaces

Java

```java
static <T> void printAll(T[] items) {        // <T> before the return type
    for (T item : items) System.out.print(item + " ");
}

interface GenericInterface<T> { void display(T value); }
class Main implements GenericInterface<Integer> {
    public void display(Integer value) { System.out.println(value); }
}
```

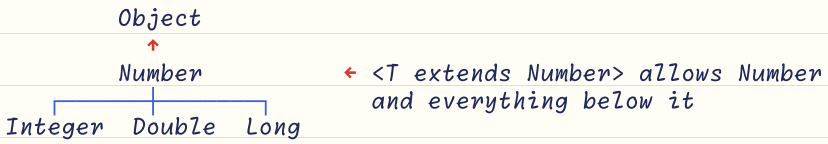
✦ BOUNDS & WILDCARDS ✦

🧠 Bounded type parameter — `<T extends Number>`

WildcardExample

```
public class WildcardExample<T extends Number> { // T: Number or subclass
    // inside, T has Number's methods: item.doubleValue(), item.intValue()
}

WildcardExample<Integer> a;   //✓
WildcardExample<Double>  b;   //✓
WildcardExample<String>  c;   //× String is not a Number
```



🎯 The big surprise

List is NOT a List

```
import java.util.*;

class Test {
    void f() {
        List<Number> nums = new ArrayList<Integer>();
    }
}
```

> 💥 *compile error*
>
> `error: incompatible types: ArrayList<Integer> cannot be converted to List<Number>`

Why? *If it were allowed,* `nums.add(3.14)` *would push a* `Double` *into a list of* `Integer`*s. Wildcards are the safe way to say "a list of some kind of Number":*

syntax	*means*	*read items as*	*can add?*
`List<?>`	*list of some unknown type*	`Object`	× *(only* `null`*)*
`List<? extends Number>`	*Number or any* ***sub****type*	`Number`	× *(only* `null`*)*
`List<? super Integer>`	*Integer or any* ***super****type*	`Object`	✓ `Integer`*s*

Main.java

```java
import java.util.List;

public class Main {
    // accepts List<Integer>, List<Double>, List<Number> ...
    static double sum(List<? extends Number> list) {
        double total = 0;
        for (Number n : list) total += n.doubleValue();
        return total;
    }

    public static void main(String[] args) {
        System.out.println(sum(List.of(1, 2, 3)));
        System.out.println(sum(List.of(1.5, 2.5)));
    }
}
```

▶ Output

```
6.0
4.0
```

☕ JAVA REALITY

🟡 GOOD TO KNOW Memory trick **PECS** — Producer `extends`, Consumer `super`: a list you only **read** from → `? extends T`; a list you only **put** into → `? super T`.

✦ COMPARING OBJECTS ✦

? WHY?

`Arrays.sort(intArray)` is obvious. `Arrays.sort(students)` — by marks? roll no? name? Java can't guess. Two ways to tell it:

- → `Comparable` — the class defines its own *natural* order (`compareTo`)
- → `Comparator` — an outside rule you pass in (`compare`), as many as you like

`compareTo` / `compare` return **negative** (this first) · **0** (equal) · **positive** (other first).

Main.java

```java
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        Student[] list = {
            new Student(12, 89.76f), new Student(5, 99.52f),
            new Student(2, 95.52f),  new Student(13, 77.52f)
        };
        Arrays.sort(list);              // natural order → compareTo (marks)
        System.out.println(Arrays.toString(list));

        // a Comparator as a lambda → roll-number order
        Arrays.sort(list, (a, b) -> Integer.compare(a.rollno, b.rollno));
        System.out.println(Arrays.toString(list));
    }
}

class Student implements Comparable<Student> {
    int rollno;
    float marks;

    Student(int rollno, float marks) {
        this.rollno = rollno;
        this.marks = marks;
    }

    @Override
    public int compareTo(Student o) {
        return Float.compare(this.marks, o.marks);   // <0 · 0 · >0
    }

    @Override
    public String toString() { return rollno + ":" + marks; }
}
```

▶ Output

```
[13:77.52, 12:89.76, 2:95.52, 5:99.52]
[2:95.52, 5:99.52, 12:89.76, 13:77.52]
```

💻 The same Comparator — anonymous class → lambda

Java

```
Arrays.sort(list, new Comparator<Student>() {       // anonymous inner class
    @Override
    public int compare(Student o1, Student o2) {
        return -(int) (o1.marks - o2.marks);        // descending
    }
});

// the same Comparator, written as a lambda:
Arrays.sort(list, (o1, o2) -> -(int) (o1.marks - o2.marks));
```

🎯 INTERVIEW TRAP #28

`(int) (o1.marks - o2.marks)` **truncates**: 99.52 − 99.1 = 0.42 → `(int)` → **0** → "equal" → wrong order! Use `Float.compare(o2.marks, o1.marks)` (descending) — never subtract-and-cast floats.

✦ COMPARABLE vs COMPARATOR ✦

	Comparable<T>	Comparator<T>
Package	`java.lang`	`java.util`
Method	`int compareTo(T other)`	`int compare(T a, T b)`
Lives	**inside** the class	**outside** — separate class or lambda
Orders per class	one natural order	unlimited
Must edit the class?	✓ yes	× no — works for classes you can't touch
Used by	`Arrays.sort(arr)`, `Collections.sort(list)`, `TreeSet`	`Arrays.sort(arr, cmp)`, `list.sort(cmp)`
JDK examples	`String` (A→Z), `Integer` (1→9)	`Comparator.reverseOrder()`, `String.CASE_INSENSITIVE_ORDER`

```
Comparable  =  "I know how to compare MYSELF"   (built into Student)
Comparator  =  "a JUDGE from outside"   (byMarks, byRollNo, byName …)
```

⚠ COMMON MISTAKE

Sorting an array of objects that implement **neither** → compiles, then crashes at runtime:

`ClassCastException: Student cannot be cast to class java.lang.Comparable`.

✦ LAMBDA EXPRESSIONS ✦

🧠 IDEA

A lambda is a **method without a name** that you can store in a variable and pass around — behaviour as data.

```
(parameters) -> expression

(parameters) -> {
                  statements;
                  return value;
                }
```

```
(a, b) -> a + b

(a, b) -> {
            int r = a - b;
            return r;
          }
```

📖 DEFINITION

A lambda implements a functional interface — an interface with **exactly one abstract method**: `Runnable`, `Comparator<T>`, `Consumer<T>`, `Function<T, R>`, `Predicate<T>`, or your own. `@FunctionalInterface` makes the compiler check the "exactly one" rule.

💻 EXAMPLE – LambdaFunctions

LambdaFunctions.java

```java
import java.util.ArrayList;
import java.util.function.Consumer;

public class LambdaFunctions {
    public static void main(String[] args) {
        ArrayList<Integer> arr = new ArrayList<>();
        for (int i = 0; i < 5; i++) arr.add(i + 1);

        Consumer<Integer> fun = item -> System.out.print(item * 2 + ",");
        arr.forEach(fun);                          // pass behaviour IN
        System.out.println();

        Operation sum  = (a, b) -> a + b;
        Operation prod = (a, b) -> a * b;
        Operation sub  = (a, b) -> {               // block body
            int result = a - b;
            return result;
        };

        LambdaFunctions calc = new LambdaFunctions();
        System.out.println(calc.operate(5, 3, sum));
        System.out.println(calc.operate(5, 3, prod));
        System.out.println(calc.operate(5, 3, sub));
    }

    private int operate(int a, int b, Operation op) {
        return op.operation(a, b);
    }
}

@FunctionalInterface
interface Operation {
    int operation(int a, int b);       // exactly ONE abstract method
}
```

▶ *Output*

```
2,4,6,8,10,
8
15
2
```

❓ WHY are lambdas useful?

- 1 line instead of a 5-line anonymous class (compare the Comparator page)
- pass **behaviour** into methods: `forEach`, `sort`, `removeIf`, streams
- one `operate()` method, unlimited operations → the calculator never changes

🎯 INTERVIEW TRAP #29

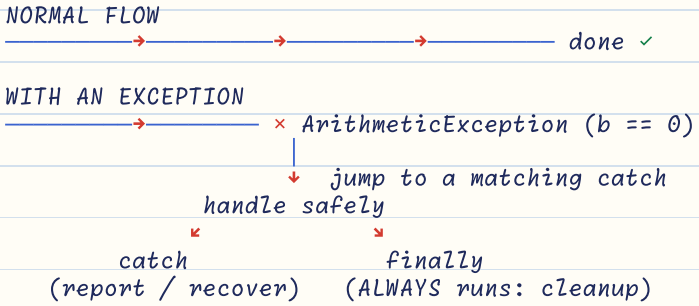
Local variables used inside a lambda must be **final or effectively final**:

`int x = 1; x++; Runnable r = () -> System.out.println(x);` × "local variables referenced from a lambda expression must be final or effectively final".

✦ EXCEPTION HANDLING ✦

🧠 IDEA

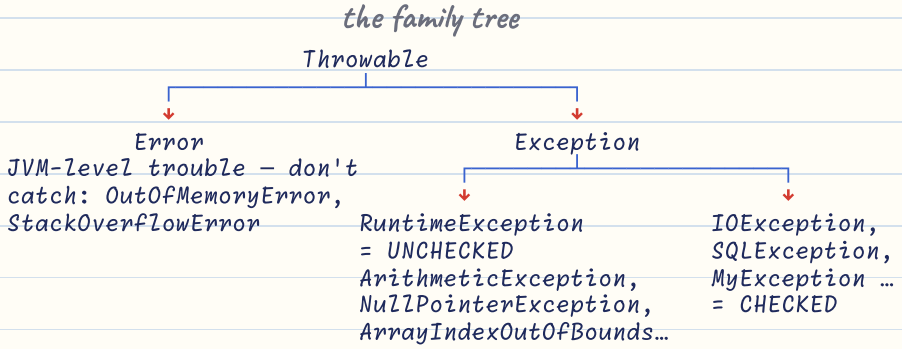
An **exception** is an unexpected event at runtime that breaks the normal flow — divide by zero, a `null`, a missing file. Handle it, and the program survives.



🌍 IN REAL LIFE

Driving to college (normal flow) → a tyre bursts (exception). Unhandled → the journey ends (crash). Handled → put on the spare (`catch`) — and whatever happens, lock the car at the end (`finally`).

✧ `catch` = plan B
`finally` = lock the car, always



💻 EXAMPLE

Main.java

```java
public class Main {
    public static void main(String[] args) {
        try {
            String name = "Kunal";
            if (name.equals("Kunal")) {
                throw new MyException("name is kunal");    // raise it NOW
            }
        } catch (MyException e) {
            System.out.println(e.getMessage());
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        } catch (Exception e) {                    // most general → LAST
            System.out.println("normal exception");
        } finally {
            System.out.println("this will always execute");
        }

        try {
            divide(5, 0);
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
    }

    static int divide(int a, int b) throws ArithmeticException {
        if (b == 0) {
            throw new ArithmeticException("please do not divide by zero");
        }
        return a / b;
    }
}

class MyException extends Exception {        // a CHECKED custom exception
    public MyException(String message) {
        super(message);                              // message → getMessage()
    }
}
```

▶ Output

```
name is kunal
this will always execute
please do not divide by zero
```

⭐ JAVA RULE

- catch **specific → general**; reversed → × "exception ArithmeticException has already been caught"
- `finally` runs whether or not an exception happened — even after `return` (only `System.exit()` or a JVM crash skips it)
- a `try` needs at least one `catch` **or** a `finally`
- one catch for several types: `catch (IOException | SQLException e)`

☕ JAVA REALITY

For files/connections use **try-with-resources** — `try (var in = new FileReader("a.txt")) { … }` closes the resource automatically, even on exceptions. That's the modern replacement for `finalize()` (Ch 01).

✦ throw vs throws ✦

	throw	throws
What	a **statement** that raises an exception right now	a **declaration** warning callers it might happen
Where	inside a method body	in the method signature
Followed by	**one object**: `throw new X("msg")`	exception **class names**, comma-separated
How many	one at a time	several: `throws IOException, SQLException`
Analogy	actually throwing the ball ⚾	the sign "beware: balls may fly"

Java

```java
// throws = the warning label on the signature
static int divide(int a, int b) throws ArithmeticException {
    if (b == 0) throw new ArithmeticException("no / by 0");  // throw = act
    return a / b;
}
```

🧠 Exception propagation

Not caught where it's thrown? It travels **down the call stack** until someone catches it — or the JVM prints a stack trace and stops.

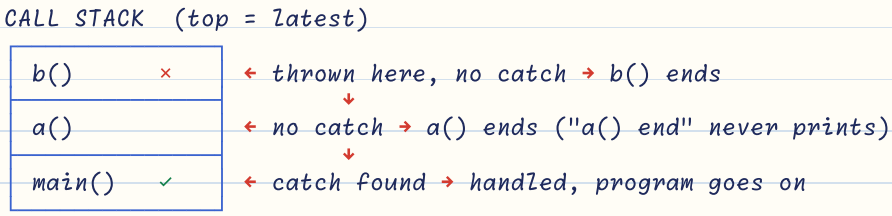
Main.java

```java
public class Main {
    public static void main(String[] args) {
        try {
            a();
        } catch (ArithmeticException e) {
            System.out.println("caught in main: " + e.getMessage());
        }
        System.out.println("program continues");
    }

    static void a() { b(); System.out.println("a() end"); }   // skipped!
    static void b() { System.out.println(10 / 0); }           // thrown here
}
```

▶ Output

```
caught in main: / by zero
program continues
```


CALL STACK (top = latest)
b() ×
a()
main() ✓
thrown here, no catch → b() ends
no catch → a() ends ("a() end" never prints)
catch found → handled, program goes on

✦ CHECKED vs UNCHECKED ✦

	CHECKED	UNCHECKED
Extends	`Exception` (but not `RuntimeException`)	`RuntimeException` (or `Error`)
Compiler forces handling?	✓ *catch it* **or** *declare* `throws`	× *no*
Usual cause	*the outside world: files, network, DB*	*bugs in our code: null, bad index, ÷0*
Examples	`IOException`, `FileNotFoundException`, `SQLException`, `CloneNotSupportedException`, `InterruptedException`	`ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ClassCastException`, `NumberFormatException`
Ignore it →	× *compile error*	*compiles; crashes if it happens*

MyException not handled

```
class MyException extends Exception {
    MyException(String m) { super(m); }
}

class Demo {
    void f() {
        throw new MyException("oops");      // neither caught nor declared
    }
}
```

💥 *compile error*

```
error: unreported exception MyException; must be caught or declared to be thrown
```

✍️ Custom exceptions — pick the parent

Java

```
class MyException extends Exception { … }                    // checked
class InvalidMarksException extends RuntimeException { … }    // unchecked
```

🎯 INTERVIEW TRAP #30

"Checked exceptions happen at compile time" × — **all** exceptions happen at runtime. Checked only means the **compiler checks** that you handled or declared them.

✦ OBJECT CLONING ✦

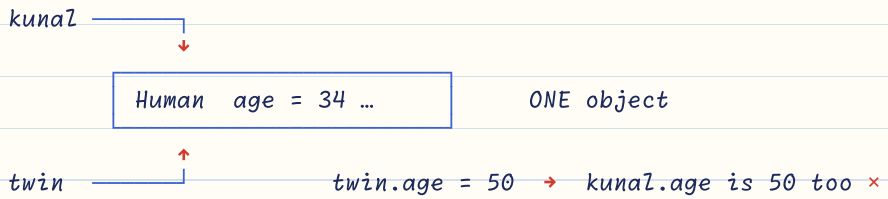
❓ WHY?

We want a **twin**: a separate object with the same data, so changing the twin doesn't touch the original.

① Reference copy — NOT a copy at all

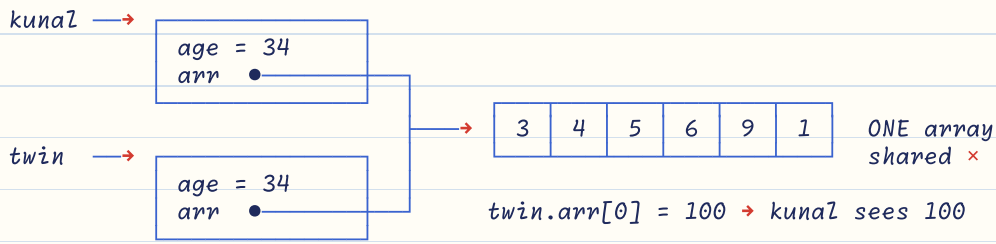
Java

```
Human kunal = new Human(34, "Kunal Kushwaha");
Human twin = kunal;              // copies the ARROW only (Ch 01!)
```

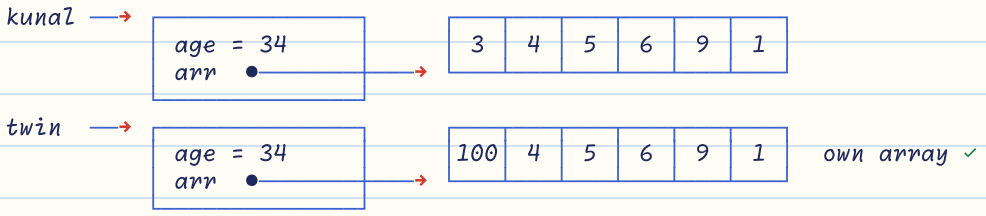


② Shallow copy — `super.clone()`

New object ✓, every field copied **as-is**: primitives by value, references by **arrow** → nested objects are **shared**.



③ Deep copy — clone and copy what's inside



💻 EXAMPLE — deep copy

Main.java

```java
import java.util.Arrays;

public class Main {
    public static void main(String[] args)
            throws CloneNotSupportedException {
        Human kunal = new Human(34, "Kunal Kushwaha");
        Human twin = (Human) kunal.clone();
        twin.arr[0] = 100;
        System.out.println(Arrays.toString(twin.arr));
        System.out.println(Arrays.toString(kunal.arr));    // untouched ✓
    }
}

class Human implements Cloneable {       // marker interface: "clone allowed"
    int age;
    String name;
    int[] arr = {3, 4, 5, 6, 9, 1};

    Human(int age, String name) { this.age = age; this.name = name; }

    @Override
    public Object clone() throws CloneNotSupportedException {
        Human twin = (Human) super.clone();    // shallow: fields as-is
        twin.arr = new int[this.arr.length];   // deep: twin's OWN array
        for (int i = 0; i < arr.length; i++) twin.arr[i] = this.arr[i];
        return twin;
    }
}
```

▶ Output

```
[100, 4, 5, 6, 9, 1]
[3, 4, 5, 6, 9, 1]
```

Keep only `return super.clone();` → **both** lines print `[100, 4, 5, 6, 9, 1]`.

⭐ JAVA RULE

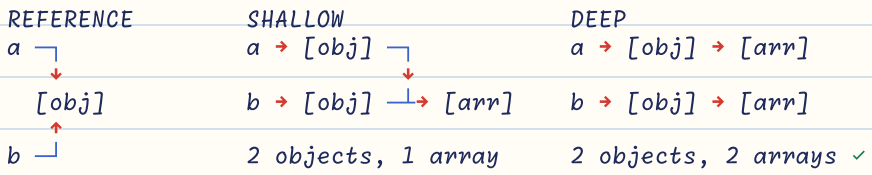
- the class must `implements Cloneable` (a **marker** interface — no methods) or `super.clone()` throws `CloneNotSupportedException`
- `Object.clone()` is `protected` → override it as `public` to call it from outside
- `super.clone()` builds the copy **without running any constructor**

☕ JAVA REALITY

Many Java experts avoid `Cloneable` (awkward design: checked exception, casts, shallow by default). A **copy constructor** — `new Human(kunal)` — is often clearer. Sharing a `String` field is fine: strings are immutable.

✦ SHALLOW COPY vs DEEP COPY ✦

	REFERENCE COPY	SHALLOW COPY	DEEP COPY
Code	`b = a;`	`super.clone()`	clone + copy nested objects
New object?	×	✓	✓
Primitive fields	same object, so shared	copied	copied
Nested objects / arrays	shared	× **shared**	✓ copied
`b.arr[0] = 100` **changes** `a`**?**	✓ yes	✓ **yes**	× no
Cost	free	cheap	more time & memory



🎯 INTERVIEW TRAP #31

"`clone()` gives an independent copy" × — by default it's **shallow**. Every mutable field (arrays, lists, other objects) must be copied by hand for a deep copy.

⭐ CHAPTER 06 – REMEMBER

⭐ REMEMBER

- ★ custom ArrayList = array + `size` + private `resize()` (double + copy)
- ★ generics = compile-time type safety, no casts; objects only; erased at runtime
- ★ `<T extends Number>` bounds T · `List<Integer>` is not a `List<Number>` · use wildcards
- ★ `Comparable` = natural order inside · `Comparator` = outside rules, often lambdas
- ★ lambda implements a functional interface (exactly one abstract method)
- ★ catch specific → general · `finally` always runs · `throw` acts, `throws` warns
- ★ checked = compiler forces handling · shallow clone shares nested objects

🧪 PRACTICE – CHAPTER 06

🧪 QUICK CHECK

1) Why can't you write `new T[10]` inside a generic class?
2) `List<Number> nums = new ArrayList<Integer>();` — does it compile? Why (not)?
3) A method contains `try { return 1; } finally { System.out.println("F"); }`. You print its result — what appears?

💻 CODE PRACTICE

1) Generic `Pair<K, V>` with getters, plus `static <T extends Comparable<T>> T max(T a, T b)`. Test with `Integer`s and `String`s.
2) A checked `InvalidMarksException`; `Student.setMarks()` throws it outside 0–100; `main` catches and prints the message. Then make it unchecked — what changes in the code?

🎯 INTERVIEW THINKING

A `Document` with a `List<String> lines` implements `Cloneable` and simply `return super.clone();`. The app's "undo" keeps a clone before each edit. Why does undo mysteriously stop working?

☑ Answers

1) Type erasure — at runtime T is unknown, so the JVM can't create a `T[]`; use `Object[]` + cast. 2. ✗ — otherwise a `Double` could be added to a list of `Integer`s; use `List<? extends Number>`. 3. `F` then `1` — `finally` runs before the method actually returns. **Interview:** shallow copy — the "backup" and the live document **share the same** `lines` **list**, so every edit changes the backup too. Deep-copy the list: `copy.lines = new ArrayList<>(lines);`.

👨‍🏫 HOW I WOULD TEACH THIS

1) Build the int ArrayList — then ask: "now store Strings?"
2) Labelled box → generics; show the compile error for a wrong type
3) Sort students: `Comparable`, then a `Comparator` lambda
4) Lambda = "a nameless method you can pass around"
5) Tyre-burst story → run `divide(5, 0)` with and without `try`
6) Cloning: draw the shared array, change it live

🔗 NEXT →

We built **one** list by hand. Java ships a whole family — lists, sets, queues, maps — under one design. And for values that may only be one of a fixed set (days, sizes, states)? → **Collections & enums** (Chapter 07).

07 COLLECTIONS · VECTOR · ENUMS

why collections · the framework map · List · Set · Queue · Map · Vector & synchronization · enums

✦ WHY COLLECTIONS? ✦

❓ WHY? — what arrays can't do

- **fixed size** — decided at creation (we hacked around it with our own ArrayList in Ch 06)
- no built-in add / remove / search / "contains" / sort-for-objects — you write the loops
- one shape only: an indexed row. Real problems want unique items, first-come-first-served, look up by name …

🌍 IN REAL LIFE — different containers for different jobs

a numbered SHELF	keeps order, allows duplicates	List
a STAMP ALBUM	each stamp only once	Set
a TICKET-COUNTER LINE	first come, first served	Queue
a DICTIONARY	look up a meaning by its word	Map

📖 DEFINITION

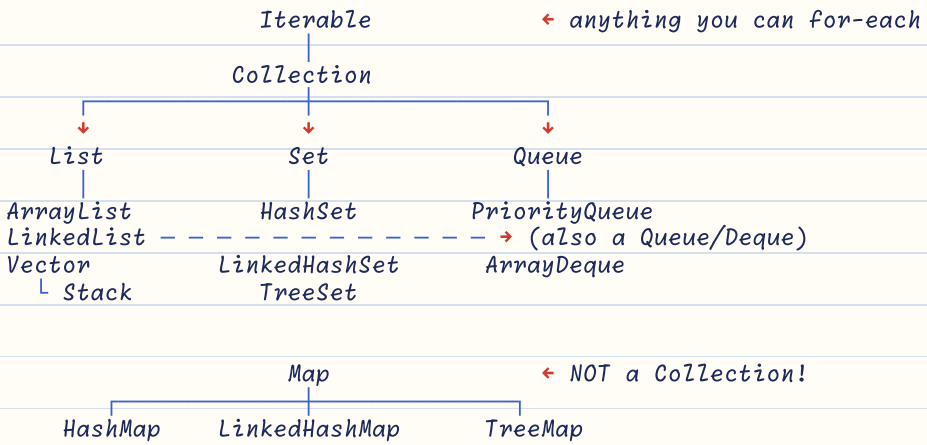
The Collections Framework (`java.util`) is a unified architecture for storing and manipulating groups of objects: **interfaces** (what a container can do), **implementations** (how it does it) and **algorithms** (`Collections.sort`, `max`, `reverse` …).

💡 CORE IDEA

It's every earlier chapter working together: **interfaces** (`List`) + **generics** (`List<Integer>`) + **polymorphism** (`List<Integer> l = new ArrayList<>()`) + **abstraction** (you never see the resize code).

✦ THE COLLECTIONS MAP ✦

MUST KNOW



interface	promise	go-to class
`List`	ordered, index-based, duplicates ✓	`ArrayList`
`Set`	no duplicates	`HashSet` (`TreeSet` = sorted)
`Queue`	FIFO: add at the back, take from the front	`ArrayDeque` , `LinkedList`
`Map`	key → value pairs, keys unique	`HashMap` (`TreeMap` = sorted keys)

✍ Program to the interface

✧ List = order
Set = unique
Queue = FIFO

Java

```
List<Integer> list = new ArrayList<>();     // ✓ reference type = interface
// later: new LinkedList<>() or new Vector<>() — nothing else changes
```

🎯 INTERVIEW TRAP #32

`Map` **does not extend** `Collection` — `Collection<String> c = new HashMap<>();` × "incompatible types". A map stores **pairs**, not single elements. (Loop it with `map.entrySet()` , `keySet()` or `values()` .)

🎯 INTERVIEW TRAP #33

`Collection` ≠ `Collections` . `Collection` is the root **interface**; `Collections` is a **utility class** of static helpers: `Collections.sort(list)` , `Collections.max(list)` , `Collections.synchronizedList(…)` .

✦ THE BIG FOUR — QUICK TOUR ✦

Main.java

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();       // order + duplicates
        list.add(34); list.add(78); list.add(34);
        list.remove(Integer.valueOf(78));             // remove by VALUE
        System.out.println(list + " first=" + list.get(0));

        Set<String> set = new HashSet<>();            // no duplicates
        set.add("java"); set.add("java"); set.add("dsa");
        System.out.println(set.size() + " " + set.contains("dsa"));

        Queue<String> queue = new LinkedList<>();     // FIFO
        queue.offer("A"); queue.offer("B"); queue.offer("C");
        System.out.println(queue.poll() + " " + queue);

        Map<String, Integer> marks = new HashMap<>();    // key → value
        marks.put("Kunal", 89);
        marks.put("Rahul", 95);
        marks.put("Kunal", 99);                       // same key → replaced
        System.out.println(marks.get("Kunal") + " " + marks.size());
    }
}
```

▶ Output

```
[34, 34] first=34
2 true
A [B, C]
99 2
```

🧠 ArrayList vs LinkedList — the picture

```
ArrayList    [ 10 | 20 | 30 | 40 |    |    ]   one block of memory
               ↑ get(2) jumps straight there → fast reads
               insert in the middle → shift everything → slow

LinkedList   (10) ⇔ (20) ⇔ (30) ⇔ (40)          separate linked nodes
               get(2) walks node by node → slow reads
               insert/remove at the ends → just relink → fast
```

🧠 Map — the picture

```
   KEY              VALUE
"Kunal" ──→          99
"Rahul" ──→          95        keys unique · values may repeat
```

🎯 INTERVIEW TRAP #34

On a `List<Integer>`, `list.remove(1)` removes the element at **index 1**, not the value 1. `List.of(10, 20, 30)` → `remove(1)` leaves `[10, 30]`. For the value use `list.remove(Integer.valueOf(1))`.

☕ JAVA REALITY

`HashSet` / `HashMap` don't keep insertion order (they order by hash buckets — Ch 04's `hashCode`!). Need insertion order → `LinkedHashSet` / `LinkedHashMap`; sorted → `TreeSet` / `TreeMap`.

✦ COLLECTION vs MAP ✦

	Collection	Map
Stores	*single elements*	***key → value** pairs*
Root of	`List`, `Set`, `Queue`	`HashMap`, `TreeMap`, `LinkedHashMap`
Extends `Iterable`?	✓ — `for (x : coll)` *works*	× — *loop over* `entrySet()` / `keySet()` / `values()`
Add	`add(e)`	`put(k, v)`
Find	`contains(e)`	`get(k)`, `containsKey(k)`
Duplicates	`List` ✓, `Set` ×	*keys* × · *values* ✓
Example	*names of all students*	*roll no. → student*

looping a Map

```
// Map.Entry = a nested interface (Ch 05)
for (Map.Entry<String, Integer> e : marks.entrySet()) {
    System.out.println(e.getKey() + " → " + e.getValue());
}
```

⭐ REMEMBER

- `Collection` = *a bag of elements* · `Map` = *a lookup table*
- `Map` *is part of the Collections* **Framework** *but not a* `Collection`
- `Map.Entry` *is the nested interface from Ch 05*

✦ VECTOR & SYNCHRONIZATION ✦

🧠 IDEA

`Vector` is a growable array — like `ArrayList` — but **every method is** `synchronized`: only **one thread at a time** can run a method on the same Vector.

Main.java

```java
import java.util.List;
import java.util.Vector;

public class Main {
    public static void main(String[] args) {
        List<Integer> vector = new Vector<>();  // same List API
        vector.add(45);
        vector.add(5);
        vector.add(15);
        vector.add(56);
        System.out.println(vector);
    }
}
```

▶ Output

```
[45, 5, 15, 56]
```

🌍 IN REAL LIFE

A shop with **one billing counter and one token**: one customer at a time — no mixed-up bills, but you wait in line even when the shop is empty (that waiting = locking cost).

```
ArrayList (not synchronized)          Vector (synchronized)
Thread 1 ─→ add() ┐                   Thread 1 ─→ add()  ✓ holds the lock
Thread 2 ─→ add() ┘→ corrupt? ×       Thread 2 ─→ add()  … waits its turn
```

	`ArrayList`	`Vector`
Synchronized	×	✓ every method
Speed, single thread	faster	slower (lock on every call)
Grows by	~50 %	100 % (doubles)
Age	Java 1.2 — part of the framework	Java 1.0 — legacy, retro-fitted to `List`
Default capacity	10 (allocated on the first `add`)	10
Today	the default choice	rarely used

☕ JAVA REALITY

🟡 GOOD TO KNOW Need a thread-safe list today? `Collections.synchronizedList(new ArrayList<>())` or `CopyOnWriteArrayList` (`java.util.concurrent`). And remember: synchronized **methods** don't make **combinations** safe — `if (!v.contains(x)) v.add(x);` can still race between the two calls.

✦ ENUMS ✦

❓ WHY? — the problem

risky

```
String day = "Mondy";       // typo — compiles happily
int size = 7;               // 7 means...? nothing stops invalid values
```

🧠 IDEA

An **enum** is a type with a **fixed** set of named values. The compiler allows only those.

🌍 IN REAL LIFE

A coffee menu: SMALL, MEDIUM, LARGE — you can't order "MEDIUM-ish". A traffic light: RED, YELLOW, GREEN — nothing else.

📖 DEFINITION

An `enum` is a special class whose instances are a fixed list of `enum constants` — each one `public static final`, created once by the JVM.

💻 EXAMPLE — Basic.java

Basic.java

```
public class Basic {
    interface A { void hello(); }

    enum Week implements A {
        Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday;
        // each constant = a public static final Week object

        Week() {                                    // implicitly private
            System.out.println("Constructor called for " + this);
        }

        @Override
        public void hello() { System.out.println("hey how are you"); }
    }

    public static void main(String[] args) {
        Week week = Week.Monday;        // first use → ALL 7 constants built
        week.hello();
        System.out.println(Week.valueOf("Monday") + " " + week.ordinal());
    }
}
```

▶ *Output*

```
Constructor called for Monday
Constructor called for Tuesday
Constructor called for Wednesday
Constructor called for Thursday
Constructor called for Friday
Constructor called for Saturday
Constructor called for Sunday
hey how are you
Monday 0
```

🧠 VISUALIZE IT — what the compiler builds

```
enum Week { Monday, Tuesday, … }

≈  final class Week extends java.lang.Enum<Week> {
       public static final Week Monday  = new Week();   // ordinal 0
       public static final Week Tuesday = new Week();   // ordinal 1
       …
       private Week() { … }
   }
```

method	returns	example
`values()`	*all constants, in order*	`for (Week d : Week.values())`
`valueOf(String)`	*the constant with **exactly** that name*	`Week.valueOf("Monday")`
`ordinal()`	*position from 0*	`Week.Wednesday.ordinal()` → *2*
`name()` / `toString()`	*the name*	`"Monday"`
`compareTo()`	*order by ordinal*	`Monday.compareTo(Friday)` < *0*

switch on an enum

```
switch (week) {
    case Saturday, Sunday -> System.out.println("weekend");
    default               -> System.out.println("work day");
}
```

🎯 INTERVIEW TRAP #35

`valueOf` *is* ***case-sensitive:*** `Week.valueOf("monday")` → `IllegalArgumentException: No enum constant Basic.Week.monday`.

✦ ENUM CONSTRUCTORS, METHODS & INHERITANCE ✦

💻 Constants with their own data

Main.java

```
public class Main {
    enum Size {
        SMALL(100), MEDIUM(150), LARGE(200);   // each calls the constructor

        private final int price;               // every constant has its own

        Size(int price) { this.price = price; }

        int getPrice() { return price; }
    }

    public static void main(String[] args) {
        for (Size s : Size.values()) {
            System.out.println(s + " " + s.ordinal()
                               + " price=" + s.getPrice());
        }
        Size mine = Size.valueOf("LARGE");
        System.out.println(mine == Size.LARGE);    // == is safe for enums
    }
}
```

▶ Output

```
SMALL 0 price=100
MEDIUM 1 price=150
LARGE 2 price=200
true
```

⭐ JAVA RULE

Constructors - implicitly `private`; writing `public` / `protected` × "modifier public not allowed here" - run **once per constant**, when the enum class is first used — never by you: `new Week()` × "enum classes may not be instantiated" - why? public constructors would allow extra objects — against the whole idea of a fixed set

🧬 Enums and inheritance

```
java.lang.Enum          ← every enum extends this, silently
      ↑
    Week   - - - implements - - →  A  (interfaces ✓, many)
      ↑
  ×  class X extends Week → "cannot inherit from final"
```

- an enum **can't extend** any class — it already extends `java.lang.Enum` (single inheritance!)
- an enum **can't be extended** — it's implicitly `final`
- an enum **can implement** any number of interfaces ✓
- `toString()` returns the constant's name; compare with `==` or `equals()` — same result

☕ JAVA REALITY

Two classic notes, updated: 1. "Enums can contain only concrete methods" — an enum **may** declare an `abstract` method if **every** constant supplies its own body: `PLUS { int apply(int a, int b) { return a + b; } }`. 2. "Enums can't be declared inside a method" — true before Java 16; since Java 16 **local enums** are allowed.

🎯 INTERVIEW TRAP #36

The constructor output appears at the **first use** of the enum (`Week.Monday`), not at `Week week;` — and it runs for **all** constants at once, in declaration order.

⭐ CHAPTER 07 – REMEMBER

⭐ REMEMBER

- Collection → List (order, dups) · Set (unique) · Queue (FIFO); Map is separate
- program to the interface: `List<T> x = new ArrayList<>()`
- `HashSet` / `HashMap` depend on `equals` + `hashCode` (Ch 04)
- `Vector` = synchronized legacy ArrayList — safe per call, slower
- enum = fixed set of `public static final` objects; private constructors; `values()`, `valueOf()`, `ordinal()`
- enums extend `java.lang.Enum` → can't extend, can't be extended, can implement

🧪 PRACTICE – CHAPTER 07

🧪 QUICK CHECK

1) `List<Integer> l = new ArrayList<>(List.of(10, 20, 30)); l.remove(1);` — what's left?
2) Does `Map` extend `Collection`? How do you loop through a map's pairs?
3) An enum has 4 constants and you use it 100 times. How many times does its constructor run?

💻 CODE PRACTICE

1) Word counter: split a sentence, count each word in a `HashMap<String, Integer>`, print words that appear more than once.

2) `enum TrafficLight { RED(30), YELLOW(5), GREEN(25) }` with a `duration` field and `next()` (RED → GREEN → YELLOW → RED). Simulate 6 changes.

🎯 INTERVIEW THINKING

"Vector is synchronized, so `if (!v.contains(x)) v.add(x);` is thread-safe." True or false?

☑ Answers

1) `[10, 30]` — `remove(int)` removes by **index**. 2. × — use `for (Map.Entry<K, V> e : map.entrySet())`. 3. **4** times — once per constant, at first use. **Interview:** false. Each call is atomic, but the pair isn't — another thread can add `x` between `contains` and `add`. Wrap both in one `synchronized (v) { … }` block (or use a `Set`).

👨‍🏫 HOW I WOULD TEACH THIS

1) Array pain: fixed size, manual searching
2) Containers: shelf · stamp album · ticket line · dictionary
3) Draw the map — circle "Map is NOT a Collection"
4) Swap `ArrayList` → `LinkedList` → `Vector` in one line
5) Enum: coffee sizes; show the compile error for a typo
6) Ask: when do enum constructors run?

🔗 THE END OF THE PLAYLIST →

From one `class Student` to collections of objects and type-safe constants — that's the full OOP journey. Next pages: the **interview bank**, **3 mini projects**, the **master map** and the **one-page cheat sheet**.

revise

INTERVIEW · PROJECTS · REVISION

85 interview questions · 3 mini projects · master map · one-page cheat sheet

✦ INTERVIEW BANK ✦

85 questions in 5 rounds. Answer **out loud** first, then read the answer. Numbers continue across pages so you can say "revise Q47".

🟢 FUNDAMENTALS	15	definitions you must say in one breath
🟡 INTERMEDIATE	20	rules + the "why" behind them
🔴 TRICKY	20	where most candidates slip
💻 OUTPUT	15	predict, then open the answer
🧠 CONCEPTUAL	15	design-level thinking

🟢 FUNDAMENTALS

Q1 What is OOP? Name its four pillars.

→ A way of organising programs around **objects** that bundle data and behaviour. Pillars: **inheritance, polymorphism, encapsulation, abstraction**.

Q2 Class vs object?

→ A class is the blueprint (logical, written once). An object is an instance created at runtime with `new`; it occupies heap memory and has its own state.

Q3 What exactly does `new` do?

→ Allocates memory for the object on the heap at runtime, initialises fields (defaults → initialisers → constructor) and returns a **reference** to it.

Q4 What is a constructor, and how is it different from a method?

→ A block that initialises a new object: same name as the class, no return type, runs automatically once per `new`, is never inherited and can't be `static` / `final` / `abstract`.

Q5 When does Java give you a default constructor?

→ Only when the class declares **no constructor at all**. Write any constructor and the no-arg one disappears.

Q6 What is `this`?

→ A reference to the current object. Uses: `this.field` (vs a parameter), `this(...)` (constructor chaining), passing or returning the current object.

Q7 What is a static variable?

→ A class-level variable: one shared copy, created when the class loads, accessed as `ClassName.var` (e.g. `Human.population`).

Q8 **What is a package and why use one?**

→ A namespace + folder for related classes. It prevents name clashes (two `Greeting` classes), organises code and acts as an access boundary.

Q9 **What is inheritance? Which keyword?**

→ A child class acquiring a parent's members with `extends` — an **IS-A** relationship (`BoxWeight` is a `Box`).

Q10 **What is method overloading?**

→ Same method name, different parameter lists (number / type / order). The compiler picks the version → compile-time polymorphism.

Q11 **What is method overriding?**

→ A subclass redefining an inherited instance method with the **same signature**. The JVM picks the version from the object's type → runtime polymorphism.

Q12 **What is encapsulation?**

→ Bundling data with the methods that operate on it **and** controlling access — private fields reached only through validating methods.

Q13 **What is abstraction?**

→ Exposing only what an object does while hiding how — via simple public methods, abstract classes and interfaces.

Q14 **Name Java's access modifiers from most to least restrictive.**

→ `private` → default (package-private) → `protected` → `public`.

Q15 **What is an interface?**

→ A contract of methods a class promises to implement. Methods are implicitly `public abstract` (plus `default` / `static` / `private` with bodies), fields are `public static final`; a class can implement many.

🟡 INTERMEDIATE

Q16 Why can't a static method use instance variables directly?

→ Instance variables belong to an object, and a static method runs with **no current object** (`this` doesn't exist). Use a reference: `obj.field`.

Q17 What does `super()` do, and where must it be?

→ Calls the parent's constructor; it must be the **first statement**. If you write neither `this(...)` nor `super(...)`, the compiler inserts `super()`.

Q18 In what order do constructors run in `A ← B ← C` when you write `new C()`?

→ Calls climb up to `Object`, bodies finish top-down: `Object` → `A` → `B` → `C`. The parent part must be ready before the child's.

Q19 What is dynamic method dispatch?

→ Resolving a call to an overridden method **at runtime** from the actual object's class — the mechanism behind runtime polymorphism.

Q20 List the rules for a valid override.

→ Same name & parameters; same or covariant return type; same or **wider** access; no new or broader checked exceptions; the parent method must not be `final`, `static` or `private`.

Q21 Can you overload `main()`?

→ Yes — but the launcher calls only the entry-point form (`public static void main(String[] args)`; Java 25 also accepts instance `main`). Other overloads are ordinary methods.

Q22 What is a static block and when does it run?

→ Code that runs **once** when the class is initialised (first use) — for the main class, before `main()` starts. Used for complex static initialisation.

Q23 Inner class vs static nested class?

→ Inner (non-static) objects are tied to an outer object — created as `outer.new Inner()` — and can use its instance members. Static nested classes need no outer object — `new Outer.Nested()` — and see only static outer members directly.

Q24 How do you write a singleton?

→ `private` constructor + `private static` instance + `public static getInstance()`. Make it thread-safe with eager init, synchronisation, or simply `enum Singleton { INSTANCE }`.

Q25 What does `final` mean on a variable, a method and a class?

→ Variable → assigned once. Method → can't be overridden. Class → can't be extended (`String`, wrapper classes).

Q26 **Can an abstract class have a constructor? Can you instantiate it?**

→ It can have constructors (run via `super(...)` to set up the parent part), but `new AbstractClass()` is a compile error.

Q27 **Abstract class vs interface — the two differences that matter most?**

→ (1) An abstract class can hold **state** and **constructors**; an interface can't. (2) A class extends **one** class but implements **many** interfaces.

Q28 **What are default methods and why were they added?**

→ Interface methods with a body (Java 8). They let interfaces gain new methods without breaking every class that already implements them (e.g. `forEach`).

Q29 **A class implements two interfaces that both have `default void hi()`. What happens?**

→ Compile error ("types ... are incompatible") until the class overrides `hi()`; inside it may call `A.super.hi()`.

Q30 **Why must `hashCode()` be overridden together with `equals()`?**

→ The contract says equal objects must have equal hash codes. `HashSet` / `HashMap` find the bucket by `hashCode()` first — break it and lookups miss, duplicates appear.

Q31 **`==` vs `equals()` for Strings?**

→ `==` checks whether both references point to the **same object**; `equals()` compares the **characters**. Always use `equals()` for content.

Q32 **What are generics, and what do they buy you?**

→ Type parameters (`List<T>`) — compile-time type safety, no casts, one reusable implementation for all types.

Q33 **What is type erasure?**

→ The compiler removes generic type information after checking it; at runtime `T` becomes `Object` (or its bound). Hence no `new T[]`, no `instanceof T`.

Q34 **Checked vs unchecked exceptions?**

→ Checked (subclasses of `Exception`, not `RuntimeException`) must be caught or declared with `throws`. Unchecked (`RuntimeException`, `Error`) aren't enforced by the compiler.

Q35 **Shallow vs deep copy?**

→ Shallow copies each field's value — nested objects are **shared**. Deep copy also duplicates the nested mutable objects so the copies are fully independent.

TRICKY

Q36 **Can we override a static method?**

→ No. A same-signature static method in the child **hides** the parent's; calls are resolved from the reference type at compile time, and `@Override` on it is a compile error.

Q37 **Can we override a private method?**

→ No — private methods aren't inherited. A same-named method in the child is a brand-new, unrelated method.

Q38 **Can a constructor be `final`, `static` or `abstract`?**

→ None of them — compile error. Constructors aren't inherited (so `final`/`abstract` are meaningless) and they initialise an instance (so `static` makes no sense).

Q39 **Is Java pass-by-reference for objects?**

→ No — Java is **always pass-by-value**. For objects the value copied is the reference: you can mutate the shared object, but re-pointing the parameter never affects the caller (the `swap(Integer, Integer)` puzzle).

Q40 **`final StringBuilder sb = new StringBuilder("a"); sb.append("b");` — legal?**

→ ✓ Legal. `final` freezes the **reference**, not the object; only `sb = new StringBuilder()` would fail.

Q41 **`Human h = null; h.population` where `population` is static — NullPointerException?**

→ No — it prints the value. Static access is bound to the reference's **type** at compile time; the null object is never touched.

Q42 **A parent constructor calls a method that the child overrides. Which version runs, and what does it see?**

→ The **child's** version (dynamic dispatch works even during construction) — but the child's fields aren't initialised yet, so it sees default values (`0`, `null`). See Q61.

Q43 **Can an interface have a constructor or instance fields?**

→ No. Interfaces have no state: fields are implicitly `public static final` constants, and constructors are not allowed.

Q44 **Can an abstract class be `final`?**

→ No — "illegal combination of modifiers": abstract needs a subclass, final forbids one.

Q45 **Does `import java.util.*;` import `java.util.function.Function`?**

→ No. Sub-packages are separate packages; a wildcard import covers one package only.

Q46 **Can a top-level class be `private` or `static`?**

→ *Neither. Top-level classes are `public` or package-private; `static` applies only to nested classes.*

Q47 ***Overloads `f(long)`, `f(Integer)`, `f(Object)` — which one does `f(5)` call?***

→ *`f(long)`. Resolution order: exact/widening primitive first, then boxing, then varargs. (`f('a')` also picks `long`.) See Q59.*

Q48 ***Can you `new` an enum? Can an enum extend a class?***

→ *No and no. Constants are created by the JVM; every enum already extends `java.lang.Enum`. It **can** implement interfaces.*

Q49 ***What does `try { return 1; } finally { return 2; }` return?***

→ *`2` — a `return` in `finally` overrides the earlier one (and would even swallow an exception). Never return from `finally`.*

Q50 ***`try { return x; } finally { x = 2; }` with `int x = 1` — returned value?***

→ *`1`. The return value is evaluated and saved before `finally` runs; changing the local afterwards doesn't affect it. See Q65.*

Q51 ***Is `List<String>` a subtype of `List<Object>`?***

→ *No — generics are **invariant**. Otherwise you could add an `Integer` through the `List<Object>` view. Use `List<?>` / `List<? extends Object>`.*

Q52 ***Does `super.clone()` call a constructor?***

→ *No. `Object.clone()` allocates a field-by-field copy without running any constructor (and throws `CloneNotSupportedException` unless the class implements `Cloneable`).*

Q53 ***Can a class be abstract without any abstract method?***

→ *Yes — it simply can't be instantiated. (The reverse isn't allowed: an abstract method forces the class to be abstract.)*

Q54 ***Inside `two.Derived extends one.Base`, does `new Base().display()` compile if `display()` is protected?***

→ *No. Outside the package, protected members are reachable only **through inheritance** — on `this` or references of type `Derived` (or its subtypes).*

Q55 ***`System.out.println(null);` — what does it print?***

→ *Nothing — it doesn't compile: "reference to println is ambiguous" (`println(char[])` vs `println(String)`).*

💻 OUTPUT QUESTIONS — predict first!

Q56 Reference juggling.

▶ predict the output

```
public class Main {
    public static void main(String[] args) {
        Student a = new Student();
        a.name = "A";
        Student b = a;
        b.name = "B";
        a = new Student();
        a.name = "C";
        System.out.println(b.name + " " + a.name);
    }
}

class Student { String name; }
```

☑ Answer

▶ Output

```
B C
```

`b = a` copies the arrow → "B" overwrites "A" in the shared object. Then `a` is re-pointed to a **new** object; `b` still holds the old one.

Q57 Static counter with ids.

▶ predict the output

```
public class Main {
    static int count = 0;
    int id;

    Main() { count++; id = count; }

    public static void main(String[] args) {
        Main x = new Main(), y = new Main(), z = new Main();
        System.out.println(x.id + " " + y.id + " " + z.id
                           + " " + Main.count);
    }
}
```

☑ Answer

▶ Output

```
1 2 3 3
```

`count` is shared and keeps growing; each object snapshots it into its **own** `id`.

Q58 Static blocks, instance blocks and constructors across inheritance.

▶ predict the output

```
class A {
    static { System.out.print("SA "); }
    { System.out.print("IA "); }
    A() { System.out.print("CA "); }
}

class B extends A {
    static { System.out.print("SB "); }
    { System.out.print("IB "); }
    B() { System.out.print("CB "); }
}

public class Main {
    public static void main(String[] args) {
        new B();
        System.out.println("|");
        new B();
        System.out.println("|");
    }
}
```

☑ Answer

▶ Output

```
SA SB IA CA IB CB |
IA CA IB CB |
```

*Static parts run **once** (parent class first). For every object: parent's instance block + constructor, then the child's.*

Q59 Overload resolution — widening vs boxing.

▶ predict the output

```
public class Main {
    static void f(long x)    { System.out.println("long"); }
    static void f(Integer x) { System.out.println("Integer"); }
    static void f(Object x)  { System.out.println("Object"); }

    public static void main(String[] args) {
        f(5);
        f('a');
        f(Integer.valueOf(5));
        f("hi");
    }
}
```

☑ Answer

▶ Output

```
long
long
Integer
Object
```

Widening (`int` / `char` → `long`) beats boxing. An `Integer` matches exactly. A `String` only fits `Object`.

Q60 Fields vs overridden methods.

▶ predict the output

```
class Parent {
    String name = "Parent";
    String getName() { return name; }
}

class Child extends Parent {
    String name = "Child";
    @Override String getName() { return name; }
}

public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        System.out.println(p.name + " " + p.getName()
                           + " " + ((Child) p).name);
    }
}
```

☑ Answer

▶ Output

```
Parent Child Child
```

*Fields follow the **reference** type (no polymorphism); the overridden method follows the **object**; the cast changes the reference type to `Child`.*

Q61 An overridden method called from the parent's constructor.

▶ predict the output

```
class Parent {
    Parent() { show(); }
    void show() { System.out.println("Parent show"); }
}

class Child extends Parent {
    int x = 5;
    @Override void show() { System.out.println("Child show " + x); }
}

public class Main {
    public static void main(String[] args) {
        new Child();
    }
}
```

☑ Answer

▶ Output

```
Child show 0
```

Dynamic dispatch picks `Child.show()` — but `Parent()` runs **before** `Child`'s field initialisers, so `x` is still `0`.

Q62 Static hiding vs overriding.

▶ predict the output

```
class A {
    static String who() { return "A"; }
    String me() { return "A"; }
}

class B extends A {
    static String who() { return "B"; }
    @Override String me() { return "B"; }
}

public class Main {
    public static void main(String[] args) {
        A obj = new B();
        System.out.println(obj.who() + obj.me() + B.who());
    }
}
```

☑ Answer

▶ Output

```
ABB
```

`obj.who()` → static, reference type `A`. `obj.me()` → overridden, object `B`. `B.who()` → called on class `B`.

Q63 The string pool.

▶ predict the output

```java
public class Main {
    public static void main(String[] args) {
        String s1 = "java";
        String s2 = "ja" + "va";          // joined by the compiler
        String part = "ja";
        String s3 = part + "va";          // joined at runtime
        System.out.println((s1 == s2) + " " + (s1 == s3)
                           + " " + s1.equals(s3));
    }
}
```

☑ Answer

▶ Output

```
true false true
```

Constant expressions are folded at compile time → same pooled object. Runtime concatenation creates a ***new*** *String;* `equals` *still matches the characters.*

Q64 The Integer cache.

▶ predict the output

```java
public class Main {
    public static void main(String[] args) {
        Integer a = 100, b = 100, c = 1000, d = 1000;
        System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));
    }
}
```

☑ Answer

▶ Output

```
true false true
```

Autoboxing reuses cached objects for −128…127; 1000 creates two separate objects. `equals` *compares values.*

Q65 `finally` vs the returned value.

▶ predict the output

```java
public class Main {
    static int test() {
        int x = 1;
        try {
            return x;
        } finally {
            x = 2;
            System.out.println("finally x=" + x);
        }
    }

    public static void main(String[] args) {
        System.out.println(test());
    }
}
```

☑ Answer

▶ Output

```
finally x=2
1
```

`return x` saves the value **1** first; `finally` runs before the method actually leaves, but changing `x` can't alter the saved value.

Q66 Which catch wins?

▶ predict the output

```java
public class Main {
    public static void main(String[] args) {
        try {
            int[] arr = new int[2];
            arr[2] = 5;
            System.out.println("A");
        } catch (ArithmeticException e) {
            System.out.println("B");
        } catch (RuntimeException e) {
            System.out.println("C");
        } finally {
            System.out.println("D");
        }
        System.out.println("E");
    }
}
```

☑ Answer

▶ Output

```
C
D
E
```

`ArrayIndexOutOfBoundsException` skips "A", isn't an `ArithmeticException`, is a `RuntimeException` → "C". `finally` always → "D". Handled, so the program continues → "E".

Q67 Pass-by-value with an array.

▶ predict the output

```java
public class Main {
    static void change(int[] arr, int n) {
        arr[0] = 99;              // changes the SHARED array
        n = 99;                   // changes a local copy
        arr = new int[]{-1};      // re-points the local copy only
    }

    public static void main(String[] args) {
        int[] arr = {1, 2};
        int n = 1;
        change(arr, n);
        System.out.println(arr[0] + " " + n);
    }
}
```

☑ Answer

▶ Output

```
99 1
```

The method received a **copy of the reference**: mutating through it is visible, re-pointing it isn't. `n` is a primitive copy.

Q68 Shallow clone.

▶ predict the output

```
public class Main {
    static class Box implements Cloneable {
        int size = 1;
        int[] items = {1, 2};

        public Box clone() throws CloneNotSupportedException {
            return (Box) super.clone();
        }
    }

    public static void main(String[] args) throws Exception {
        Box a = new Box();
        Box b = a.clone();
        b.size = 5;
        b.items[0] = 9;
        System.out.println(a.size + " " + a.items[0] + " " + (a == b));
    }
}
```

☑ Answer

▶ Output

```
1 9 false
```

*Two different objects (`false`); the primitive `size` was copied; the `items` array is **shared** by the shallow copy.*

Q69 `remove(int)` vs `remove(Object)`.

▶ predict the output

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 2));
        Set<Integer> set = new HashSet<>(list);
        list.remove(1);
        list.remove(Integer.valueOf(2));
        System.out.println(list + " " + set.size());
    }
}
```

☑ Answer

▶ Output

```
[1, 3] 3
```

*`remove(1)` deletes **index** 1 → `[1, 3, 2]`; `remove(Integer.valueOf(2))` deletes the first value 2 → `[1, 3]`. The set dropped the duplicate: {1, 2, 3}.*

Q70 When do enum constructors run?

▶ predict the output

```
public class Main {
    enum Level {
        LOW, MEDIUM, HIGH;
        Level() { System.out.print(name().charAt(0)); }
    }

    public static void main(String[] args) {
        System.out.print("start ");
        Level l = Level.HIGH;
        System.out.println(" " + l.ordinal() + " "
                           + Level.LOW.compareTo(l));
    }
}
```

☑ Answer

▶ Output

```
start LMH 2 -2
```

All constants are built at the enum's **first use**, in order (L, M, H). `HIGH` has ordinal 2; `compareTo` subtracts ordinals: 0 − 2 = −2.

🧠 CONCEPTUAL

Q71 Why does Java forbid multiple inheritance of classes but allow many interfaces?

→ Two parent **classes** can bring conflicting code and duplicate state (the diamond problem) plus two constructor chains. Interfaces carry no state, and conflicting defaults must be resolved explicitly — so many interfaces are safe.

Q72 Why is `main()` static?

→ At startup no object exists; `static` lets the JVM call `main` on the class itself without guessing how to construct one. (Java 25's launcher can also create the object for an instance `main`, but the classic answer stands.)

Q73 Why aren't constructors inherited?

→ A constructor initialises **its own** class's part of the object. A parent's constructor knows nothing about the child's fields, so inheriting it would create half-initialised objects. Children reuse it explicitly through `super(...)`.

Q74 Why is composition often preferred over inheritance?

→ HAS-A relationships can be swapped at runtime (`NiceCar.upgradeEngine()`), don't lock you into one parent, and don't break when a base class changes (the fragile base class problem). Use inheritance for true IS-A families.

Q75 When would you choose an abstract class over an interface?

→ When related classes share **state and code** (fields, constructors, protected helpers) and form a genuine family. Choose an interface for a capability that unrelated classes can implement.

Q76 Compile-time type vs runtime type — why does the difference matter?

→ The compile-time (reference) type decides **what you may call**; the runtime (object) type decides **which overridden method runs**. Every polymorphism question is really this sentence.

Q77 Why keep fields private even if the getter and setter do nothing special?

→ Freedom to change later: add validation, make the field read-only, compute it, log changes — without touching any caller. Public fields freeze your design forever.

Q78 How does a `HashMap` find a key?

→ `hashCode()` picks the bucket; `equals()` finds the exact key inside it. That's why the two must agree (Ch 04).

Q79 Why are Strings immutable (and the class final)?

→ Safe sharing in the string pool, cacheable hash codes for map keys, thread safety, and security (file paths, class names can't be altered after checks). `final` stops subclasses from breaking those

guarantees.

Q80 **What do generics solve that an `Object`-based container can't?**

→ With `Object` the compiler can't stop wrong types going in, and every read needs a cast that may fail at runtime. Generics move those errors to compile time and remove the casts.

Q81 **Conceptually, why can't static methods be overridden?**

→ Overriding means "decide at runtime from the object". A static method has no object — it's bound to the class at compile time — so there is nothing to dispatch on.

Q82 **When should you create a custom exception — checked or unchecked?**

→ When a domain-specific failure deserves its own name (`InsufficientBalanceException`). Checked if the caller can reasonably recover and must be forced to think about it; unchecked for programming errors.

Q83 **Why is `Vector` considered legacy?**

→ It predates the Collections Framework and synchronises every call — a cost most code doesn't need, and it still doesn't make compound actions safe. `ArrayList` + targeted concurrency tools are better.

Q84 **Why are enums better than `int` or `String` constants?**

→ Type safety (only valid values compile), a namespace, usable in `switch`, can carry fields and methods, built-in `values()` / `valueOf()`, and `==` comparison is safe.

Q85 **How do the four pillars cooperate in one design — say, payments?**

→ An interface `PaymentMethod` (**abstraction**) implemented by `Card`, `Upi`, `Wallet` (**inheritance** of type); `checkout(PaymentMethod p)` calls `p.pay()` (**polymorphism**); each class keeps balances and PINs private behind validated methods (**encapsulation**).

✦ PROJECT 1 · REPORT CARD SYSTEM ✦

Ch 01 Ch 02 Ch 03 classes · objects · constructors · `this` · static · encapsulation

📝 Problem statement

A coaching institute keeps students' marks for 5 subjects in **three parallel arrays** (the exact pain from Ch 01) — and last week someone typed `140` as a mark. Build a small OOP system that stores students, **never accepts invalid data**, and prints report cards.

✅ Requirements

1) Every `Student` gets an automatic, **read-only** roll number 1, 2, 3 … (a static counter).
2) Two constructors — `Student(String name)` and `Student(String name, int[] marks)` — chained with `this(...)`.
3) `boolean setMarks(int subject, int value)` accepts only subject 0–4 and value 0–100.
4) `percentage()` and `grade()` → A ≥ 90 · B ≥ 75 · C ≥ 60 · D otherwise.
5) `toString()` returns one report-card line.
6) `static int totalStudents()`.
7) `ReportCardApp.main`: 3 students, 2 invalid updates (rejected), all cards, the topper.

🧩 Classes & relationships

```
ReportCardApp ──creates──▶ Student  (many objects)
                           ├─ static int count        ◀ shared
                           ├─ final int rollNo        ◀ read-only
                           ├─ private String name
                           ├─ private int[] marks (5)
                           ├─ Student(name) ─this(...)─▶ Student(name, marks)
                           ├─ boolean setMarks(...)   ◀ the ONLY way in
                           ├─ percentage() · grade()
                           └─ toString()
```

▶ Expected behaviour

sample run

```
× rejected: subject 2 → 140
× rejected: subject 7 → 50
Roll 1 | Aman  | 92.4% | A
Roll 2 | Riya  | 78.0% | B
Roll 3 | Kabir | 55.0% | D
Topper: Aman (92.4%) · students created: 3
```

Starter structure

Student.java — starter (compiles, logic is yours)

```
public class Student {
    private static int count = 0;
    private final int rollNo;
    private final String name;
    private final int[] marks = new int[5];

    public Student(String name) {
        this(name, new int[5]);              // chaining is done for you
    }

    public Student(String name, int[] marks) {
        this.rollNo = ++count;
        this.name = name;
        // TODO: copy each mark through setMarks() so validation applies
    }

    public boolean setMarks(int subject, int value) {
        // TODO: reject a bad subject index or a value outside 0..100
        return false;
    }

    public double percentage() { return 0; }   // TODO
    public char grade()        { return 'D'; } // TODO
    public int getRollNo()     { return rollNo; }
    public static int totalStudents() { return count; }

    @Override
    public String toString() {
        return "";  // TODO: "Roll 1 | Aman | 92.4% | A"
    }
}
```

⭐ Stretch goals

- implement `Comparable<Student>` and print a merit list (Ch 06)
- add a `Subject` **enum** instead of indexes 0–4 (Ch 07)
- prove `count` is shared: print `Student.totalStudents()` after each `new`

✦ PROJECT 2 · RIDEX FARE ENGINE ✦

Ch 03 Ch 05 *inheritance · polymorphism · abstraction · interfaces · overloading · overriding*

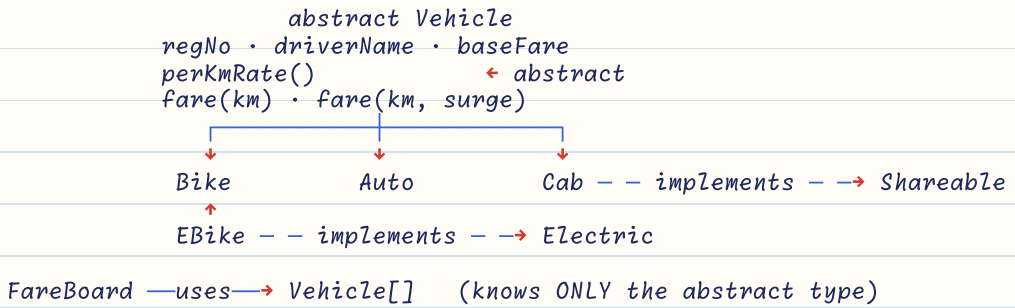
📝 Problem statement

*RideX offers **Bikes, Autos and Cabs**. Every type prices a trip differently; electric vehicles get a green discount and cabs can be **shared** (fare split between riders). The booking screen must show the fare of **every** vehicle with **one loop** — and adding a new type tomorrow (say `EBike`) must not change that loop at all.*

✅ Requirements

1) `abstract class Vehicle`: `regNo` *(final)*, `driverName`, `baseFare`; `abstract double perKmRate()`; a ***final*** `fare(double km) = baseFare + km × perKmRate()`; *an* ***overload*** `fare(double km, double surge)`.
2) `Bike`, `Auto`, `Cab` *extend* `Vehicle`, *override* `perKmRate()` *and* `toString()`; `Cab` *has* `seats`.
3) `interface Electric` — *constant* `GREEN_DISCOUNT_PERCENT = 10`, `batteryLevel()`, `default double discounted(double fare)`.
4) `interface Shareable` — `maxRiders()`, `default double splitFare(double fare, int riders)`.
5) `EBike extends Bike implements Electric`; `Cab implements Shareable`.
6) `FareBoard.showFares(Vehicle[] fleet, double km)` *uses polymorphism +* `instanceof` *patterns — it never names* `Bike`, `Auto` *or* `Cab`.
7) `new Vehicle(...)` *must be impossible; a vehicle's* `regNo` *can never change.*

🧩 Classes & relationships



▶ Expected behaviour (10 km)

sample run

```
Bike   KA-01-1234 (Ravi)   ₹ 75.00
EBike  KA-01-5678 (Asha)   ₹ 67.50   green -10 %
Auto   KA-02-1111 (Manoj)  ₹ 145.00
Cab    KA-03-2222 (Sara)   ₹ 230.00  shared by 4 → ₹ 57.50 each
Surge ×1.5 on the Cab:     ₹ 345.00
```



Starter structure

Vehicle.java — starter

```java
public abstract class Vehicle {
    private final String regNo;
    protected final String driverName;
    protected final double baseFare;

    protected Vehicle(String regNo, String driverName, double baseFare) {
        this.regNo = regNo;
        this.driverName = driverName;
        this.baseFare = baseFare;
    }

    protected abstract double perKmRate();          // each type decides

    public final double fare(double km) {           // one formula for all
        return baseFare + km * perKmRate();
    }

    public double fare(double km, double surge) { // overload
        return 0;  // TODO
    }

    public String getRegNo() { return regNo; }
}

class Bike extends Vehicle {                        // worked example
    Bike(String regNo, String driver) { super(regNo, driver, 25); }
    @Override protected double perKmRate() { return 5; }
}

interface Electric {
    int GREEN_DISCOUNT_PERCENT = 10;
    int batteryLevel();
    default double discounted(double fare) {
        return fare;  // TODO: apply GREEN_DISCOUNT_PERCENT
    }
}

interface Shareable {
    int maxRiders();
    // TODO: default double splitFare(double fare, int riders)
}

// TODO: Auto (base 45, ₹ 10/km), Cab (base 80, ₹ 15/km, Shareable), EBike
```

FareBoard.java — starter

```
public class FareBoard {
    static void showFares(Vehicle[] fleet, double km) {
        for (Vehicle v : fleet) {
            double fare = v.fare(km);              // runtime polymorphism
            // TODO: v instanceof Electric e → fare = e.discounted(fare)
            // TODO: v instanceof Shareable s → also print the split fare
            System.out.printf("%-6s %s  ₹ %.2f%n",
                    v.getClass().getSimpleName(), v.getRegNo(), fare);
        }
    }

    public static void main(String[] args) {
        Vehicle[] fleet = { new Bike("KA-01-1234", "Ravi") };  // TODO: more
        showFares(fleet, 10);
    }
}
```

⭐ Stretch goals

- add `EBike` **without editing** `showFares` — that's polymorphism paying rent
- sort the fleet by fare with a `Comparator` lambda (Ch 06)
- explain in one line why `fare(km)` is `final` but `perKmRate()` is `abstract`

✦ PROJECT 3 · LIBRARY LITE ✦

Ch 06 Ch 07 generics · collections · exceptions · enums · lambdas · OOP design

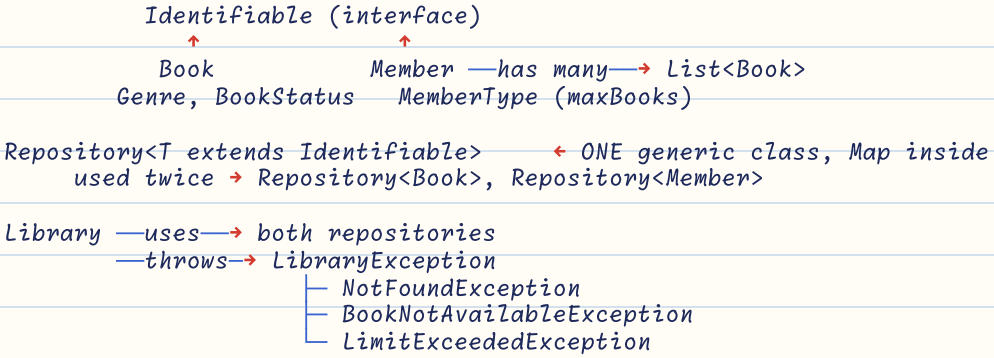
Problem statement

A college library tracks **books** and **members**. Students may borrow at most **2** books, faculty **5**. Issued or lost books can't be issued again. The librarian wants search by genre, a title-sorted list, per-genre counts — and **clear messages instead of crashes**.

Requirements

1) Enums: `Genre { FICTION, SCIENCE, HISTORY, TECH }`, `BookStatus { AVAILABLE, ISSUED, LOST }`, `MemberType { STUDENT(2), FACULTY(5) }` with a `maxBooks` field.
2) `interface Identifiable { String getId(); }`
3) `Book implements Identifiable` — id, title, author, genre, status; `equals` / `hashCode` on **id**; `toString`.
4) `Member implements Identifiable` — id, name, type, `List<Book> borrowed`.
5) Generic `Repository<T extends Identifiable>` backed by a `Map<String, T>`: `add(T)`, `T findById(String) throws NotFoundException`, `List<T> findAll(Predicate<T> filter)`.
6) Checked exceptions under one parent: `LibraryException` ← `NotFoundException`, `BookNotAvailableException`, `LimitExceededException`.
7) `Library.issue(memberId, bookId)` / `giveBack(...)` throw the right exception; `main` catches each one and prints its message — the program **never** crashes.
8) Reports: books of a genre (lambda filter), all books sorted by title (`Comparator`), count per genre (`EnumMap<Genre, Integer>`).

Classes & relationships



▶ Expected behaviour

> *sample run*
>
> ```
> Issued "Clean Code" to Aman
> Issued "Sapiens" to Aman
> × Aman (STUDENT) can borrow at most 2 books
> × Book B9 not found
> × "Clean Code" is ISSUED
> Returned "Clean Code"
> TECH books: [Clean Code, Effective Java]
> Sorted: [A Brief History of Time, Clean Code, Effective Java, Sapiens]
> Per genre: {FICTION=0, SCIENCE=1, HISTORY=1, TECH=2}
> ```

Starter structure

Library.java — starter (one file)

```
import java.util.*;
import java.util.function.Predicate;

enum Genre { FICTION, SCIENCE, HISTORY, TECH }
enum BookStatus { AVAILABLE, ISSUED, LOST }

enum MemberType {
    STUDENT(2), FACULTY(5);
    private final int maxBooks;
    MemberType(int maxBooks) { this.maxBooks = maxBooks; }
    int maxBooks() { return maxBooks; }
}

interface Identifiable { String getId(); }

class LibraryException extends Exception {
    LibraryException(String msg) { super(msg); }
}
class NotFoundException extends LibraryException {
    NotFoundException(String msg) { super(msg); }
}
// TODO: BookNotAvailableException, LimitExceededException

class Repository<T extends Identifiable> {
    private final Map<String, T> items = new HashMap<>();

    void add(T item) { items.put(item.getId(), item); }

    T findById(String id) throws NotFoundException {
        T item = items.get(id);
        // TODO: throw new NotFoundException(...) when item is null
        return item;
    }

    List<T> findAll(Predicate<T> filter) {
        List<T> result = new ArrayList<>();
        // TODO: keep every value for which filter.test(value) is true
        return result;
    }
}

// TODO: class Book implements Identifiable  (equals + hashCode on id!)
// TODO: class Member implements Identifiable
// TODO: class Library { issue(...), giveBack(...), reports }
```

Stretch goals

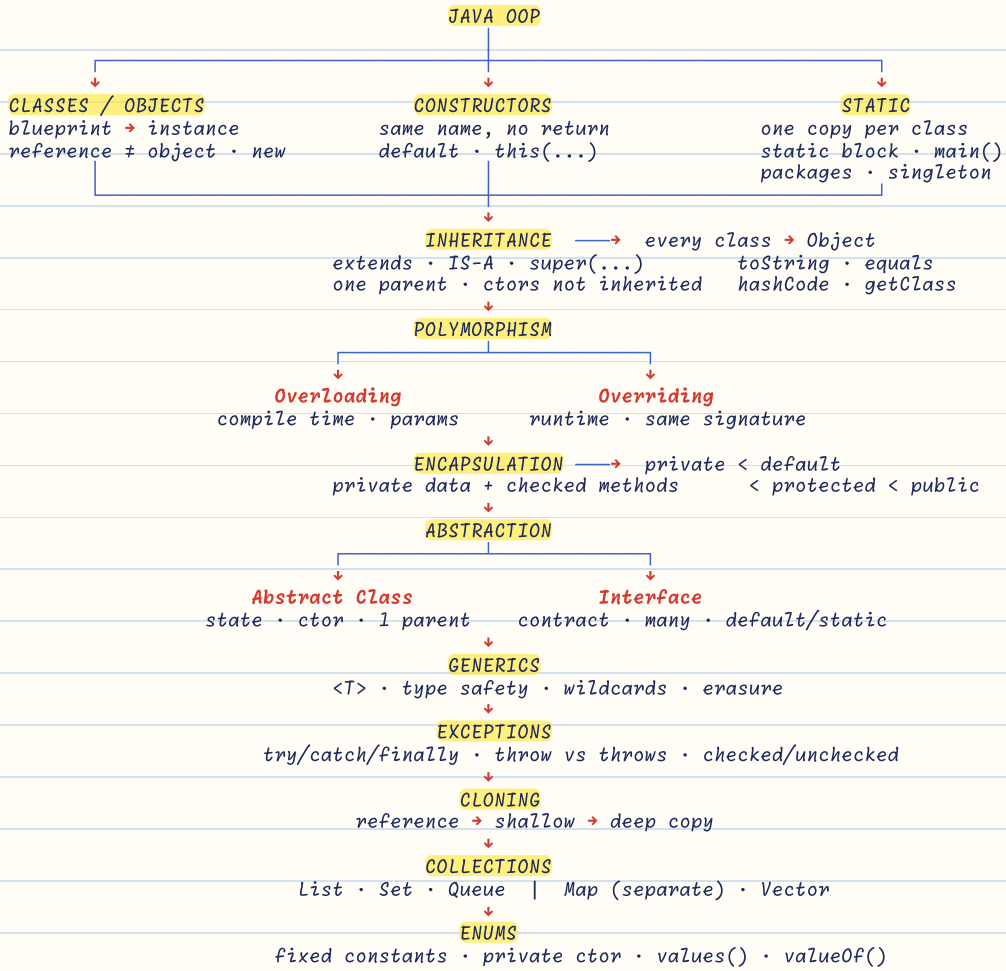
- make `LOST` books reportable with `values()` and a `switch` on `BookStatus`
- replace the `HashMap` with a `TreeMap` — what changes in the output order, and why?
- write one JUnit-style test per exception (Ch 05: annotations like `@Test`)

👨‍🏫 HOW I WOULD TEACH THIS

1) Students read the problem and **list nouns** → classes, **verbs** → methods
2) Draw the relationship sketch before any code
3) Starter first: make it compile, then fill one TODO at a time
4) Demo the expected run; ask which chapter each line comes from
5) Review: "where is encapsulation? polymorphism? generics?"

✦ THE MASTER MAP ✦

One page, the whole playlist. Cover the right side with your hand and explain each box aloud — if you can, you're ready.



💡 CORE IDEA

Read it top-down as a story: ***make objects*** *→* ***share & organise*** *them →* ***reuse*** *them (inheritance) →* ***vary*** *them (polymorphism) →* ***protect*** *them (encapsulation) →* ***simplify*** *them (abstraction) → make them* ***type-safe*** *(generics),* ***robust*** *(exceptions),* ***copyable*** *(cloning),* ***groupable*** *(collections) and* ***restricted to valid values*** *(enums).*

✦ ONE-PAGE CHEAT SHEET ✦

Keywords

keyword	meaning
`class`	defines a blueprint
`new`	creates an object, returns its reference
`this`	current object · `this(...)` same-class ctor
`super`	parent part · `super(...)` parent ctor
`static`	belongs to the class, one copy
`final`	assign once · no override · no extend
`extends`	inherit one class · an interface extends interfaces
`implements`	sign one or more interface contracts
`abstract`	no instance (class) · no body (method)
`interface`	pure contract type
`try`	guard risky code
`catch`	handle a matching exception
`finally`	always runs → cleanup
`throw`	raise one exception object now
`throws`	declare possible exceptions in the signature

The 4 pillars

pillar	in one line	Java tool
Inheritance	reuse a parent, IS-A	`extends` , `super`
Polymorphism	one call, many behaviours	overloading · overriding
Encapsulation	data + methods + controlled access	`private` + getters/setters
Abstraction	show what, hide how	abstract class · interface

Comparisons in one line

pair	the difference
Class vs Object	blueprint vs instance in the heap
Reference vs Object	the arrow vs the thing it points to
Static vs Non-static	one shared copy vs one per object
Overloading vs Overriding	new params, compile time vs same signature, runtime
Encapsulation vs Abstraction	protect the data vs hide the complexity
Abstract class vs Interface	state + ctor + 1 parent vs contract + many
`==` **vs** `equals()`	same object vs same content
Shallow vs Deep copy	nested objects shared vs copied
`throw` **vs** `throws`	raise it now vs warn in the signature
Collection vs Map	single elements vs key → value pairs

20 golden Java rules

1) A reference is not the object — `b = a` copies only the arrow.
2) Java is always pass-by-value; for objects, the reference is copied.
3) Write any constructor and the free default one disappears.
4) `this(...)` / `super(...)` come first; `super()` is added if you write neither.
5) Constructors are never inherited — so never overridden.
6) Static = one copy per class; a static context has no `this`.
7) Static methods are hidden, not overridden; fields are never polymorphic.
8) Reference type → what you may call; object type → which override runs.
9) Overloading needs different parameters — return type alone isn't enough.
10) Overriding keeps the signature and can't reduce access.
11) `final` freezes the variable, not the object it points to.
12) One parent class, many interfaces.
13) Interface fields are `public static final`; its methods are `public`.
14) Abstract classes can't be instantiated but do have constructors.
15) `protected` outside the package works only through inheritance.
16) Override `equals()` → override `hashCode()` with the same fields.
17) Compare objects with `.equals()`; `==` checks identity.

18) Generics are invariant and object-only: `List<Integer>` ≠ `List<Number>`, `List<int>` ×.

19) Catch specific → general; `finally` always runs (never `return` from it).

20) `Map` is not a `Collection`; enums can't extend or be extended.

✦ end of notebook ✦ revise the traps (#1–#36), redo the 15 output questions, build one mini project ✦